國立清華大學

資訊工程學系

專題研究成果報告

以尾數位元換瓦數：

在 GH200 功耗上限下以尾數歸零提升科學計算效能

Trading Mantissa Bits for Watts:
Software Mantissa Zeroing under Module Power Caps
on the NVIDIA GH200

學　　生： 陳克盈（Koying Chen）

學　　號： 112062205

指導教授： 周志遠 教授

中華民國 115 年 10 月

摘要

現代超級電腦與 GPU 叢集越來越常在功耗上限下運作：機房依機櫃電力限電，學生超算競賽限制整個叢集的功率，NVIDIA GH200 也允許使用者設定整顆模組的功耗上限。在功耗上限下，GPU 會把時脈降到功耗剛好不超過上限，因此程式的速度取決於每次運算消耗的能量；而 GPU 的耗電又隨資料而變，位元翻轉越多越耗電。另一方面，許多科學應用實際需要的精度，遠少於 FP64 提供的 52 個尾數位元。

本專題研究「尾數歸零」：以軟體將運算元低位的尾數位元設為 0，不改變資料格式與演算法，使運算單元與資料線路的翻轉減少，再於功耗上限下把省下的功耗轉換為更高的時脈與吞吐量。我們在 14 節點的 GH200 叢集上，以成對交錯的量測與「等效功耗」（未修改的程式需要多給多少功耗上限才能追上）評估此做法，涵蓋矩陣乘法微基準、8 節點 HPL 與 HPL-MxP、Quantum ESPRESSO 與 PySCF 等科學軟體、第三方非矩陣乘法程式，以及 AI 訓練與推論。

結果顯示，在功耗受限且截斷直接實作於程式內時，所有受測、以矩陣乘法為主且具正確性檢查的完整程式皆獲得正面收益。8 節點 HPL 保留 36 位有效位數（含隱藏位，即 35 個尾數位元；殘差距門檻約百倍）在 400–800 W 快 4.0–11.0%，相當於每節點多給 39–127 W 的功耗上限；HPL-MxP 保留 3 位快 1.8–12.7%（對角優勢的測試矩陣）；Quantum ESPRESSO 依求解器的收斂門檻自動調整位元數，在總能量印出的每一位皆不變的情況下快 2.4–4.2%、少耗電 2.5–3.9%（金表面的收益約一半來自 Davidson 迭代減少）；DF-MP2 截斷到 20–32 位在 500–700 W 快 4.9–8.3%；指數跨度寬且功耗高的高斯過程地統計截斷到 28 位在 500–700 W 快 7.4–8.2%，估出的參數與原生相同，而 FP32 結果不可用、INT8 模擬較慢。在預設的 1000 W 上限下，HPL 仍快 3.9%，每次求解少耗電 7.5%。機制實驗顯示 62–97% 的省電來自低位元不再翻轉，在 tensor core、FP64 乘加單元與記憶體搬移上皆成立。

適用範圍方面，與 INT8 模擬 FP64 的分界點隨矩陣形狀與功耗上限而變；所有矩陣乘法改用 FP32 或 3xTF32 時，長內積的 Quantum ESPRESSO 無法收斂；列內指數跨度寬時，固定位數的 INT8 模擬可能在沒有警告的情況下算錯（高斯基底 DFT 上 cuBLAS 固定 36 位模擬的能量誤差達 52 mE_h，同樣保留 36 位的最近捨入截斷只差 $10^{-6}\,E_\mathrm{h}$），但跨度寬不一定導致錯誤，以 tile 為單位換格式也能處理這類資料，截斷還可疊加其上；在已使用 FP8 的 AI 推論上，FP8 已取得幾乎全部的好處。尾數歸零每個元素只多一次位元運算、不需要新硬體，最適合具殘差修正或自洽迭代的科學求解器。

註：本報告中所有截斷後的 HPL ／ HPL-MxP 與 MLPerf 數字皆為研究用途，並非 TOP500 或 MLPerf 的正式成績。

關鍵詞：功耗上限、能源效率、浮點精度、尾數截斷、GPU、混合精度、HPL

Abstract

We study a software knob that exploits data-dependent GPU power: zeroing unneeded low-order mantissa bits of FP64, BF16 and FP8 operands. The study runs on 14 NVIDIA GH200 nodes under module power caps, with paired measurements and an equivalent-power metric (the extra cap the unmodified program needs to catch up). Under a binding cap, every tested GEMM-dominated HPC program that truncates internally and checks its results gains. At 400–800 W, tuned 8-node HPL keeping 36 significand bits (about 100× residual margin) runs 4.0–11.0% faster, worth 39–127 W per node, and HPL-MxP keeping 3 bits runs 1.8–12.7% faster on a diagonally dominant test matrix. At 450–550 W, Quantum ESPRESSO, with the bit count tied to its Davidson tolerance, runs 2.4–4.2% faster (partly from fewer iterations) with every printed energy digit unchanged. At 500–700 W, DF-MP2 keeping 20–32 bits runs 4.9–8.3% faster, and a wide-range Gaussian-process likelihood keeping 28 bits runs 7.4–8.2% faster, where FP32 fails and INT8 emulation is slower. At the default 1000 W cap, HPL is still 3.9% faster and uses 7.5% less energy per solve. Fill controls attribute 62–97% of the saving to bits that stop toggling. Safe bit counts need validation across problem sizes, and the crossover with INT8 emulation of FP64 moves with matrix shape and power cap. With wide per-row exponent spans, fixed-width INT8 emulation can fail silently: at 36 bits it shifts a Gaussian-basis DFT energy by 52 mE_h, whereas rounding to 36 bits errs by $10^{-6}\,E_\mathrm{h}$. All-FP32 or 3xTF32 GEMMs fail on long inner products, and in Hopper AI inference FP8 already captures most of the benefit. Truncated HPL, HPL-MxP and MLPerf numbers in this report are research results, not official TOP500 or MLPerf scores.

Keywords: power capping, energy efficiency, floating-point precision, mantissa truncation, GPU, mixed precision, HPL

目錄

摘要 i

Abstract ii

圖目錄 vi

表目錄 vii

符號與縮寫說明 viii

第一章 緒論 1

1.1 研究背景：功耗成為算力的上限 1
1.2 功耗隨資料而變 1
1.3 應用用不到全部的尾數 1
1.4 研究問題 2
1.5 主要貢獻 2
1.6 報告架構 3

第二章 背景知識 4

2.1 浮點格式與尾數 4
2.2 GPU 功耗與位元翻轉 4
2.3 功耗上限與時脈 5
2.4 其他以降低精度換取速度的做法 5
2.5 本報告使用的基準與應用 5

第三章 相關研究 7

3.1 資料相依的 GPU 功耗 7
3.2 功耗受限的高效能計算 7
3.3 硬體層的尾數遮罩 7
3.4 精度調校與 FP64 模擬 8

3.5 地統計的混合精度 8
3.6 依收斂程度調整精度 9
3.7 本研究的定位 9

第四章 研究方法 **10**
4.1 尾數歸零與三種捨入方式 10
4.2 四種實作層次 10
4.3 實驗平台 11
4.4 量測與統計 11
4.5 等效功耗 ΔW 12
4.6 誤差模型 12

第五章 實驗結果 **14**
5.1 省電機制：位元不翻轉或數值為零 14
5.2 功耗上限下的矩陣乘法微基準 15
5.3 HPL 與 HPL-MxP：具正確性檢查的完整基準 16
5.4 科學軟體：Quantum ESPRESSO 與 PySCF 19
5.5 與 FP64 模擬及單精度的分界 22
5.6 非矩陣乘法的 FP64 程式 25
5.7 AI 工作負載 27
5.8 高功耗的真實應用：DF-MP2 29
5.9 指數跨度：只能以截斷處理的資料 30
5.10 跨度寬且功耗高的應用：高斯過程地統計 32

第六章 討論 **36**
6.1 適用條件 36
6.2 與改變格式做法的關係 36
6.3 限制與效度威脅 38
6.4 規則合規性 39

第七章 結論與未來工作 **40**
7.1 結論 40
7.2 未來工作 41

參考文獻 **42**

附錄 A 實驗環境、程式位置與重現方式 **48**

A.1 實驗環境 48

A.2 程式位置 48

A.3 重現方式 49

圖目錄

5.1 省電機制：被捨去的位元填 0、全 1 或固定樣式時的省電 14
5.2 矩陣乘法微基準在各功耗上限下的加速 15
5.3 調參後 8 節點 HPL 的效能與功耗上限 17
5.4 8 節點 HPL-MxP 在各功耗上限下的加速 18
5.5 Quantum ESPRESSO 金表面的總能量誤差與保留位數 20
5.6 方陣 FP64 矩陣乘法：截斷、INT8 模擬與單精度的比較 23
5.7 纈胺黴素的兩種真實矩陣形狀上截斷與模擬的速度 23
5.8 截斷與 INT8 模擬的分界點掃描 24
5.9 非矩陣乘法核心在 1000 W 下的省電 26
5.10 Llama 2 70B LoRA 的驗證損失：原生與截斷 2 位 27
5.11 六個語言模型在各格式下的困惑度增加 28
5.12 指數跨度對各方法逐元素誤差的影響 30
5.13 各應用的整體指數跨度與列內小元素比例 31

表目錄

2.1 本報告使用的浮點格式 4

3.1 本研究與相關研究的比較 9

5.1 調參後 8 節點 HPL 在各功耗上限下的效能 17
5.2 HPL 縮放殘差隨保留位數與矩陣大小 N 的變化（門檻 16） 18
5.3 8 節點 HPL-MxP 在各功耗上限下的效能 19
5.4 科學軟體所需的尾數位數（攔截所有 FP64 矩陣乘法） 20
5.5 Quantum ESPRESSO 程式內截斷與自動調整位元數的時間與耗電 . . . 21
5.6 單精度做法的矩陣乘法相對誤差隨內積長度 k 的變化 25
5.7 第三方非矩陣乘法程式的係數截斷結果 26
5.8 BF16 最近捨入保留 3 位與 FP8 在 FP8 容易出問題的場景下的比較 . . 28
5.9 DF-MP2（紫杉醇，def2-TZVP）各變體的 MP2 時間與相對原生的加速 29
5.10 各應用的列內指數跨度，以及截斷與 INT8 模擬的誤差 31
5.11 高斯過程對數概似評估的加速與偏差 33
5.12 截斷與 tile 混合精度疊加的效果 34
5.13 ExaGeoStatCPP 的 tile Cholesky 分解時間比較（ABBA 兩輪平均） . . 34

6.1 各類工作負載在功耗受限時的收益 37

A.1 實驗環境與軟體版本 48
A.2 程式與資料位置 48

符號與縮寫說明

縮寫／符號	說明
3xTF32	把每個 FP32 數拆成兩個 TF32、以三次 tensor core 乘法近似 FP32 精度的做法
ABBA	成對交錯的執行順序（A、B、B、A），用來消除溫度與先後順序的影響
BF16、FP16、FP8	16 位元的 bfloat16、16 位元的 IEEE 半精度、8 位元浮點（本報告指 E4M3）
cuBLAS	NVIDIA 的 GPU 線性代數函式庫；第 13 版起內建 FP64 矩陣乘法的 INT8 模擬
DFT	密度泛函理論（density functional theory）
DF-MP2	密度擬合的二階 Møller–Plesset 微擾理論
E_h、Ry	原子單位制的能量單位 Hartree 與 Rydberg（1 E_h = 2 Ry）
FP64、FP32、TF32	IEEE 雙精度、IEEE 單精度，以及 tensor core 使用的 19 位元 TF32 格式
GEMM、DGEMM	一般矩陣乘法；FP64 的一般矩陣乘法
GEMMul8	以中國餘數定理實作 Ozaki 第二版方法的 INT8 模擬函式庫
GH200	NVIDIA Grace Hopper superchip（Grace CPU 與 Hopper GPU 封裝在同一模組）
HBM	高頻寬記憶體（high-bandwidth memory）
HPL、HPL-MxP	TOP500 排名所用的 FP64 Linpack 基準；以低精度分解加迭代修正的混合精度 Linpack 基準
INT8 模擬	把 FP64 數拆成多片 8 位元整數、以整數 tensor core 計算 FP64 矩陣乘法的做法（Ozaki 方法）
LoRA	低秩適配（low-rank adaptation）微調
MLE	最大概似估計（maximum likelihood estimation）
SCF	自洽場（self-consistent field）迭代
SM	串流多處理器（streaming multiprocessor）；本報告的「時脈」皆指 SM 時脈
k	保留的尾數位數；HPL 的位數依 rocHPL 慣例為含隱藏位的有效位數
ΔW	等效功耗：未修改的程式要追上尾數歸零版本，每個節點需要多給的功耗上限（第 4.5 節）
α	矩陣乘法誤差模型的係數（第 4.6 節）；在第 1.2 節另指電路的切換活動率

第一章　緒論

1.1　研究背景：功耗成為算力的上限

大型計算系統的電力與散熱都有預算。機房依機櫃的供電能力設定功耗上限，學生超算競賽（Student Cluster Competition, SCC）限制整個隊伍的總功率，NVIDIA GH200 superchip 也允許使用者直接設定整顆模組（GPU、CPU 與記憶體）的功耗上限（本叢集的驅動程式接受 200–1000 W，見第 4.3 節）。當程式的耗電碰到上限時，GPU 的功耗管理會降低時脈與電壓，直到功耗不超過上限。因此在功耗受限時，效能不再由硬體的最高時脈決定，而是由每次運算所消耗的能量決定：每次運算的耗電越低，時脈便能拉得越高。

1.2　功耗隨資料而變

CMOS 電路的動態功耗約為

$$P_{\mathrm{dyn}} \approx \alpha\, C\, V^2 f, \tag{1.1}$$

其中 C 為負載電容、V 為電壓、f 為時脈，α 為切換活動率：資料位元在相鄰時脈之間翻轉（0 變 1 或 1 變 0）得越多，電路充放電的次數越多，耗電就越高。因此同一個程式、同樣的運算量，輸入資料不同，耗電也不同。前人已在 A100 與 GH200 上量到，同樣的矩陣乘法，隨機輸入比固定值或全零輸入多耗 60–67% 的功率（見第三章）。

1.3　應用用不到全部的尾數

FP64 有 52 個尾數位元，約相當於 16 位十進位有效數字，但許多應用並不需要這麼高的精度：HPL 只要求殘差小於門檻；平面波密度泛函計算的自洽迭代在前期只需要很寬鬆的精度；AI 推論甚至以 FP8（只有 3 個尾數位元）也能維持準確度。這些用不到的低位元在運算中仍持續隨機翻轉，所消耗的電力並未帶來有用的精度。

1.4　研究問題

綜合以上三點，本專題的核心想法是：**以軟體將用不到的低位尾數設為 0，使其不再翻轉，並在功耗上限下把省下的功耗轉換為效能**。本專題要回答以下三個問題：

1. **機制：**為什麼少幾個尾數位元就會省電？省電來自位元不再翻轉，還是來自數值為 0？
2. **效益：**在功耗上限下，真實的科學應用能換得多少效能與省電？需要付出多少精度？
3. **適用範圍：**與直接改用較小格式的做法（INT8 模擬 FP64、FP32 ／ 3xTF32、FP8）相比，尾數歸零在哪些情況下有價值？

1.5　主要貢獻

1. **功耗上限下真實應用的收益**。以「未修改的程式需要多給多少功耗上限才能追上」的等效功耗量化收益，涵蓋附信賴區間的矩陣乘法微基準、調參後的 8 節點 HPL、HPL-MxP、Quantum ESPRESSO、高功耗的 DF-MP2，以及指數跨度寬且功耗高的高斯過程地統計，並以各程式本身的判準（殘差、總能量、估計參數）檢查結果。
2. **省電機制**。以全 1 或固定樣式取代 0 填入被捨去的位元，在 tensor core 矩陣乘法、FP64 乘加單元與記憶體搬移三個層次上，分開「位元不再翻轉」與「數值為 0」兩種效應，在 GH200 的模組功耗上量化了前人提出的翻轉假說。
3. **所需位數與失效邊界**。提出只用求解器自身容許誤差決定位元數的規則，使 Quantum ESPRESSO 的總能量印出的每一位都不變；並測試 HPL（多組問題大小）、PySCF、第三方非矩陣乘法程式、LoRA 微調與六個語言模型可以截斷到什麼程度，以及捨入方式的影響。
4. **與換格式做法的分界**。與 INT8 模擬（cuBLAS、GEMMul8）、FP32 ／ 3xTF32 及 AI 的 FP8 比較，指出分界點隨矩陣形狀、功耗上限、累加精度與指數跨度移動；列內指數跨度寬時，固定位數的 INT8 模擬可能安靜地算錯，而這個風險可以事先以低成本的逐列跨度統計偵測。

1.6　報告架構

第二章介紹浮點格式、GPU 功耗與功耗上限等背景知識；第三章整理相關研究；第四章說明研究方法與實驗平台；第五章呈現實驗結果，包括高功耗的 DF-MP2、指數跨度實驗，以及跨度寬且功耗高的高斯過程與 ExaGeoStat；第六章討論適用範圍與限制；第七章為結論與未來工作。附錄 A 列出實驗環境、程式位置與重現方式。

第二章　背景知識

2.1　浮點格式與尾數

一個浮點數由符號 s、指數 e 與尾數（fraction）f 三部分組成，正規數的數值為

$$x = (-1)^s \times 1.f \times 2^{e-\text{bias}}. \tag{2.1}$$

指數決定值域（能表示多大與多小的數），尾數決定精度（有效數字的位數）。本報告用到的格式列於表 2.1。

表 2.1　本報告使用的浮點格式

格式	符號／指數／尾數位元	約略的十進位有效位數	常見用途
FP64	1 ／ 11 ／ 52	16	科學計算、HPL
FP32	1 ／ 8 ／ 23	7	一般單精度計算
TF32	1 ／ 8 ／ 10	3	tensor core 的 FP32 加速模式
BF16	1 ／ 8 ／ 7	2–3	AI 訓練與推論
FP8 E4M3	1 ／ 4 ／ 3	1	AI 推論與訓練

本報告所稱「保留 k 位」，是指保留最高的 k 個尾數位元、將其餘位元設為 0。例如 FP64 保留 36 位，即把 52 個尾數位元中最低的 16 個清為 0。符號與指數不變，所以值域不變，只是每個值的精度變粗：直接截斷時，每個值的相對誤差不超過 2^{-k}；最近捨入時不超過 $2^{-(k+1)}$。資料格式與程式介面完全不需更動，這是它與「改用較小格式」最大的不同。

2.2　GPU 功耗與位元翻轉

依式 (1.1)，GPU 的動態功耗與切換活動率成正比。資料在暫存器、運算元匯流排、乘法器與記憶體之間流動時，只要某個位元與前一筆資料不同，對應的線路就要充電或放電。隨機資料的低位尾數幾乎每次都在翻轉；若把它們全部設為 0，相鄰兩筆資料在這些位置便永遠相同，這些線路就不再切換。在乘法器中，一個運算元的低位為 0 時，對應的部分積也恆為 0，整片電路都不會動作。記憶體方面，HBM 與 DRAM 的存取能耗同樣隨資料的翻轉率變化。

2.3　功耗上限與時脈

GH200 的模組功耗上限涵蓋 GPU、Grace CPU 與記憶體。GPU 會持續調整時脈，使功耗不超過上限（實測撞上限時的 1 秒平均功耗常低於上限，限電器依據的應是更短時間尺度的功耗）。因此會出現兩種情況：

- **功耗受限**（例如上限 500 W）：每次運算的耗電降低後，GPU 能在相同功耗下以更高的時脈運作，效果表現為**加速**。
- **不受限**（上限高於程式的用電，時脈停在最高的 1980 MHz）：速度不變，效果表現為**省電**。

1000 W 是 GH200 的預設值，也是最高上限；但功耗很高的程式（例如 FP64 矩陣乘法與 HPL）在 1000 W 下仍有部分時間撞上限，因此兩種效果會同時出現。

2.4　其他以降低精度換取速度的做法

INT8 模擬 FP64（Ozaki 方法）： 把 FP64 數拆成多片低精度整數，以 INT8 tensor core 分別相乘後再合併，片數越多越精確。cuBLAS 13 內建此類模擬；GEMMul8 實作第二版 Ozaki 方法，以中國餘數定理把數值拆到多個模數上計算。

FP32 與 3xTF32： 以單精度取代 FP64 計算。3xTF32 把每個 FP32 數拆成兩個 TF32，以三次 tensor core 乘法達到接近 FP32 的精度。

FP8： Hopper 上 AI 推論的主流格式，資料寬度為 BF16 的一半，矩陣乘法的峰值是 BF16 的兩倍，以逐張量或逐通道的縮放因子彌補較窄的值域。

混合精度迭代修正： 以低精度完成 LU 分解，再以 FP64 迭代修正到高精度；HPL-MxP 即是這類基準。

這些做法都改變資料格式，依靠吞吐量更高的運算單元加速；尾數歸零則沿用原本的運算單元，只降低輸入的翻轉。兩者各有適用場合，第 5.5 節將比較兩者的分界。

2.5　本報告使用的基準與應用

HPL： 求解稠密線性系統的 FP64 基準，為 TOP500 排名的依據。最後會檢查縮放後的殘差是否小於 16，超過即判定失敗。

HPL-MxP： 以 BF16 完成 LU 分解、再以 FP64 迭代修正的混合精度基準。

Quantum ESPRESSO： 平面波密度泛函理論軟體，以自洽場（SCF）迭代求基態，每一圈以 Davidson 方法對角化，收斂門檻逐圈收緊。

PySCF（GPU4PySCF）： 高斯基底的量子化學軟體，本報告用於 DFT、HF+MP2 與密度擬合 MP2（DF-MP2）。

高斯過程地統計： 自寫的 ExaGeoStat 式最大概似估計核心，以及 ExaGeoStatCPP（StarPU 與 Chameleon 的 tile Cholesky）。

MFEM、SW4、LAMMPS： 不以矩陣乘法為主的 FP64 程式，分別為有限元素、地震波傳播與分子動力學。

MLPerf Llama 2 70B LoRA 與六個語言模型： AI 訓練與推論。

第三章　相關研究

3.1　資料相依的 GPU 功耗

Bhalachandra 等人在 A100 上發現，同樣大小的 DGEMM，隨機運算元比固定值多耗 67% 的功率 [1]。Gregersen 等人在 A100、H100 與 V100 上改變 GEMM 輸入的分佈與位元樣式，功耗最多相差近 40%，其中把最低位元歸零可以降低功耗；但他們刻意選擇不會降頻的矩陣大小，沒有量測功耗上限下的效能 [2]。He 在 A100 上示範全零輸入的矩陣乘法比隨機輸入快約 15%，並把功耗上限從 330 W 逐步降到 100 W：可預測輸入的加速隨上限降低而變大（圖中約從 1.2 倍增加到 150 W 時的 2.3 倍），到 100 W 才轉而變小 [3]。Antepara 等人在 GH200 上量到零輸入的 DGEMM 只用 563 W，隨機輸入則用 898 W [4]。Lucas 與 Juurlink 建立的 GPU ALU 功耗模型中，主項是位元翻轉數 [5]。這些工作都以全零、常數等特殊輸入示範此現象，沒有截斷真實資料的部分尾數，也沒有評估數值精度或真實應用。

3.2　功耗受限的高效能計算

Rountree 等人最早在硬體強制的功耗上限下量測效能，發現同型處理器在同一上限下的執行時間會因製程差異而不同（約 6%） [6]；Inadomi 等人在大型系統上量到，同一功耗上限下各節點的 DGEMM 效能可相差 64%，並依此變異分配功耗 [7]。Patki 等人在固定的總功耗下配置更多節點，再以功耗上限分配電力（超量配置）[8]；Zeus 在 GPU 上依功耗上限與批次大小調整 DNN 訓練的能耗 [9]。這些工作調整的是功耗如何分配，本專題則降低每次運算所需的能量，兩者可以疊加。我們量到同一上限下節點間 13–17% 的時脈差異（第 4.4 節），也與上述觀察一致。

3.3　硬體層的尾數遮罩

以「把尾數設為 0 來省電」在硬體設計上早有前例：Tong 等人的可變位寬浮點乘法器，每次運算的能量最多可減少 66% [10]；Pool 等人為行動 GPU 的著色器選擇

精度，以能耗模型估計頂點轉換可省 23–36% 的能量 [11]，像素著色器的算術最多可省 71% [12]；Guenther 等人在基頻處理器運算單元的輸入端遮罩尾數，最多省能 23% [13]；AMD 的專利在 GPU 著色器中加入硬體歸零模組 [14]。這些做法都需要新硬體，或以模擬器估算能耗。資料搬移方面，DRAM 與 HBM 的能耗隨資料的翻轉率變化 [15–17]。

3.4　精度調校與 FP64 模擬

Precimonious 等工具自動把變數換成較低精度的型別，目標是提升速度 [18]；rpe 與 chop 以同樣的位元操作模擬低精度，但不量測功耗 [19, 20]。Transprecision computing 主張依需求在時間與空間上細粒度地調整精度 [21]；Klöwer 等人從資料本身估計真正承載資訊的尾數位數，再以最近捨入去除其餘位元，用於壓縮氣候資料 [22]。Ozaki 方法 [23] 可以在整數矩陣單元上模擬 FP64 [24, 25]，Uchino 等人評估了它的效能與功耗 [26]。cuBLAS 內建的定點模擬讓 A 的每一列共用一個縮放因子，列內的動態範圍必須由尾數位元承擔；它的自動模式依指數跨度決定切片數，所需位元超過上限時改回原生 FP64 [27,28]。形式上最接近本專題的是 Grützmacher 等人：他們把 FP64 值依尾數切段存放、保留完整指數，只讀取前面的段落，使受頻寬限制的 PageRank 快 10–30% [29]；他們截斷是為了節省頻寬，本專題則是為了降低運算的切換功耗。

3.5　地統計的混合精度

ExaGeoStat 以 FP64 的 tile Cholesky 分解計算高斯過程的概似 [30]。之後的混合精度版本先把位置依空間填充曲線排序，使強相關集中在對角線附近；對角 tile 保留 FP64，再依離對角線的距離或 tile 的範數，把整個 tile 轉成 FP32、FP16 或 FP8：FP64 ／ FP32 平均快 1.6 倍 [31]，加上 FP16 最多快 2.64 倍 [32]，結合低秩近似最多快 12 倍 [33]，在 GH200 上以四種精度比 FP64 快 3 倍 [34]，也用於 GH200 系統上的氣候模擬器 [35]。Cao 等人在單張 V100、A100 與 H100 上量測過這類分解的功耗與能耗 [36]，但沒有設定功耗上限，也沒有分析時脈。單一 FP32 不足以計算這類概似 [32]，與第 5.10 節的結果一致。這些做法都是把整個 tile 換成其他格式；本專題保留 FP64、只截斷運算元的尾數，兩者可以疊加。GPU 上的 RI-MP2 也曾以單精度與混合精度 GEMM 加速 [37]。

3.6　依收斂程度調整精度

Woo 等人在 LOBPCG 特徵值求解器中，依收斂狀態在 FP32、FP64 與混合精度之間切換 [38]；Huang 等人在 GPU4PySCF 中依 SCF 能量的變化選擇 INT8 模擬的切片數 [39]；Dawson 等人發現量子化學的密度矩陣純化約需 37 位有效尾數即已足夠 [40]。這些工作都是改用吞吐量較高的格式來加速，並未研究功耗。

3.7　本研究的定位

表 3.1 比較本研究與相關研究。據我們所知，本研究是第一個在現役 GPU（GH200）上，以純軟體截斷 FP64 ／ BF16 運算元的尾數來降低切換功耗、在功耗上限下把省下的功耗轉換為吞吐量，並在真實 HPC 應用上以物理誤差驗證的系統性研究。在 FP64 內「保留指數、捨去尾數」本身並不新 [29]，地統計的混合精度也已有人量測過功耗 [36]，全零輸入在不同功耗上限下的加速也已有人量測過 [3]；本專題的不同在於目的（降低切換功耗）、做法（不改變格式）與量測條件（功耗上限下的時脈與吞吐量）。

表 3.1　本研究與相關研究的比較

工作	需要新硬體	截斷真實資料	在功耗上限下量測效能	驗證應用的數值結果
Bhalachandra、Gregersen、Antepara	否	否（全零或特殊樣式）	否	否
He	否	否	是（330 → 100 W 掃描，全零輸入）	否
Tong、Pool、Guenther、AMD	是	是	否	部分
Woo、Huang	否	改變格式	否	是
Grützmacher（尾數切段儲存）	否	是（保留指數、捨去尾數，目的為節省頻寬）	否	是（PageRank）
ExaGeoStat 系列（Abdulah、Cao、Ren）	否	改變格式（整個 tile）	否（Cao 等人量過功耗，未設上限）	是
本研究	**否**	**是**	**是（含 8 節點 HPL）**	**是**

第四章　研究方法

4.1　尾數歸零與三種捨入方式

對一個有 p 個尾數位元的運算元（FP64 為 52、BF16 為 7、FP8 E4M3 為 3），保留最高的 k 個位元，其餘 $p - k$ 個設為 0。本研究比較三種去除低位的方式：

截斷： 直接以位元遮罩清除低位，即往 0 的方向捨去。

最近捨入： 先加上保留的最低位權重的一半（即被捨去部分的最高位），再以遮罩清除。攔截工具、GEMM 微基準、Quantum ESPRESSO、DF-MP2 與高斯過程的實作在平手時往離零較遠的方向捨入；HPL、LoRA 的權重載入與語言模型推論的實作則是平手取偶。

隨機捨入： 先加上一個小於被捨去位元權重的隨機整數，再以遮罩清除。

每個元素只需一次整數 AND 運算（最近捨入再多一次加法，隨機捨入另需產生亂數），可以融入產生該運算元的核心；本報告的 HPL 另外執行一個捨入核心、攔截工具另外做一次複製，這些額外成本都計入量到的時間。本報告所稱「截斷到 k 位」泛指保留 k 位；除非特別註明，應用程式（Quantum ESPRESSO、PySCF、DF-MP2、高斯過程、ExaGeoStat 與第三方程式）與 HPL 使用最近捨入，GEMM 微基準、分界點掃描與 HPL-MxP 使用往零截斷。HPL 的位數依 rocHPL 的慣例為含隱藏位的有效位數：36 位相當於保留 35 個尾數位元。

4.2　四種實作層次

1. **在基準程式中歸零**。矩陣乘法微基準在產生輸入時歸零；HPL 在每次 FP64 GEMM 之前以一個捨入核心處理兩個輸入（執行時以環境變數指定位數）；HPL-MxP 在編譯時指定位數，每次寫回 BF16 時歸零。

2. **攔截函式庫**。對不修改原始碼的科學軟體，以 `LD_PRELOAD` 攔下 cuBLAS 的所有 FP64 與複數 FP64 矩陣乘法入口，把 A、B 複製並捨入後再呼叫原函式（呼叫端

的資料不變）。同一個函式庫也能統計運算元的指數分佈、改用 cuBLAS 的 FP64 模擬，或改以 FP32 ／ 3xTF32 計算，用於公平比較。

3. **程式內改寫**。在 Quantum ESPRESSO 的 Davidson 對角化中，對波函數區塊原地歸零，不需要複製；第三方非矩陣乘法程式（MFEM、SW4、LAMMPS）在設定階段對儲存的係數陣列歸零一次；GPU4PySCF 的 DF-MP2 在三指標張量建好後原地歸零一次；高斯過程核心則在每個面板進行尾端更新之前原地歸零。

4. **依收斂門檻自動調整**。Davidson 每一圈會收到一個收斂門檻 ethr，在整個 SCF 過程中從約 10^{-2} 收緊到 10^{-12}。我們保留

$$k = \min(52,\ \max(16,\ \lceil -\log_2 \mathrm{ethr} \rceil + m)) \tag{4.1}$$

個位元，其中 m 為餘裕，使前期使用較少位元、最後幾圈才用足。這是一條經驗規則：ethr 是特徵值的絕對容許誤差（單位 Ry），與尾數的相對誤差不同，m 用來吸收兩者的差距；此規則不需要對應用建模，只使用程式本來就會計算的容許誤差。

LoRA 訓練則在載入後對凍結的 FP8 基底權重歸零一次；推論實驗在線性層中模擬各種格式，其中 FP8 使用真實的 FP8 tensor core 矩陣乘法。

4.3　實驗平台

實驗平台為 14 個節點的叢集，每個節點有一顆 NVIDIA GH200 480GB superchip（72 核 Grace CPU、Hopper GPU 搭配 96 GB HBM3，最高 SM 時脈 1980 MHz），以 200 Gb/s InfiniBand 連接，由 Slurm 管理。模組功耗上限可設為 200–1000 W（預設 1000 W）。規格表列出的可設範圍是 450–1000 W [41]；本報告低於 450 W 的設定超出規格表的範圍，但驅動程式接受，實測功耗也貼著上限。軟體為 NVHPC 26.5（CUDA 13.2、cuBLAS 13、NCCL、HPC-X Open MPI）。多節點實驗（HPL、HPL-MxP、LoRA）使用 8 個節點。完整的軟體版本列於附錄 A.1。

4.4　量測與統計

功耗： 每 250–500 ms 以 nvidia-smi 取樣模組功耗、SM 時脈、溫度與降頻原因。耗電量以驅動程式的 1 秒平均模組功耗對時間積分（對忽高忽低的負載，單一瞬時讀

數會有 –37% 到 +41% 的偏差）；表中的平均功耗是量測視窗內瞬時取樣的平均，穩態下與 1 秒平均相差約 1% 以內。

公平比較： 每次量測都等 GPU 降到 45 °C 以下才開始；在同一功耗上限內，以交錯順序（A、B、B、A）在同一組節點上執行各變體，以消除溫度與先後順序的影響。

信賴區間： 同一節點、同一上限、同一輪的「歸零 ÷ 原生 – 1」為一個成對樣本，以 t 分布取 95% 區間，藉此消除節點間 13–17% 的時脈差異。區間只用於矩陣乘法微基準（3 個節點各 2 對）與 Quantum ESPRESSO 的自動調整（6 對）；同一節點的兩對並非完全獨立，區間可能偏窄。HPL、HPL-MxP、DF-MP2 與高斯過程每一格只有同一組節點上的兩輪，表中列出兩輪的值或平均，不附區間。每個工作的第一次執行常偏慢（暖機），而交錯順序的第一個多半是原生，會使第一對的收益偏大（第 5.4 節）。

資料檢查： 每個量測視窗都記錄實際生效的功耗上限，與預定值不符者一律剔除（叢集為共用資源）；硬體功耗煞車以降頻計數器判讀（只在診斷工作 1675 中開啟，主要的 HPL ／ HPL-MxP 掃描沒有記錄）。

4.5　等效功耗 ΔW

在功耗上限 P 下，將尾數歸零版本的效能記為 $p'(P)$，原生版本的效能記為 $p(P)$。沿著原生版本實測的「效能對上限」曲線線性內插，找出原生版本達到 $p'(P)$ 所需的上限 P_{native}，並定義

$$\Delta W(P) = P_{\text{native}} - P, \qquad p(P_{\text{native}}) = p'(P). \tag{4.2}$$

其意義為：未修改的程式要追上尾數歸零版本，每個節點需要多給的功耗上限。這比百分比更直接地對應到機房的電力預算。上限之間以直線內插；由於效能對上限的曲線向下彎，兩個上限相隔較遠時（例如 800–1000 W），ΔW 會略為高估。

4.6　誤差模型

兩個輸入都保留 k 位時，矩陣乘法結果的相對誤差（Frobenius 範數）約為

$$\frac{\|\widehat{C} - C\|_F}{\|C\|_F} \approx \alpha \cdot 2^{-k}, \tag{4.3}$$

與內積長度無關（累加仍為 FP64）。在均勻隨機資料上（$k \geq 4$），最近捨入的 $\alpha \approx 0.27$、隨機捨入 0.38、截斷 0.70，截斷的誤差約為最近捨入的 2.6 倍。截斷的誤差幾乎全是系統性的縮小，對會累積偏差的流程有害，這是科學軟體應採用最近捨入的原因（第 5.4 節）。

第五章　實驗結果

本章依「機制、微基準、完整基準、科學軟體、與其他做法比較、非矩陣乘法程式、AI、高功耗應用、指數跨度」的順序呈現結果。除非特別說明，百分比都是與同一功耗上限、同一節點上的原生版本比較。

5.1　省電機制：位元不翻轉或數值為零

尾數歸零同時改變兩件事：低位元不再隨資料翻轉，而且被捨去部分的數值變成 0。為了分開這兩種效應，我們把被捨去的位元改填全 1，或填入一組所有元素共用的固定非零樣式；這兩種填法同樣不會翻轉，但數值不是 0。我們在 1000 W 下對三個層次各量測一次：FP64 tensor core 矩陣乘法、只使用一般 FP64 運算單元的乘加迴圈，以及純記憶體搬移（圖 5.1）。填值版本都以最高時脈 1980 MHz 執行，因此比較的是功耗；原生 FP64 矩陣乘法仍有 95% 的時間撞上限、時脈只有 1822 MHz，其功耗是被上限截住的值，所以「不翻轉」所佔的比例會算得偏保守。

三個層次的結論一致：只要讓低位元不再翻轉，就能取得完整省電的 62–97%；填 0 比填 1 或固定樣式再多省一些。記憶體搬移幾乎完全是「不翻轉」的效應

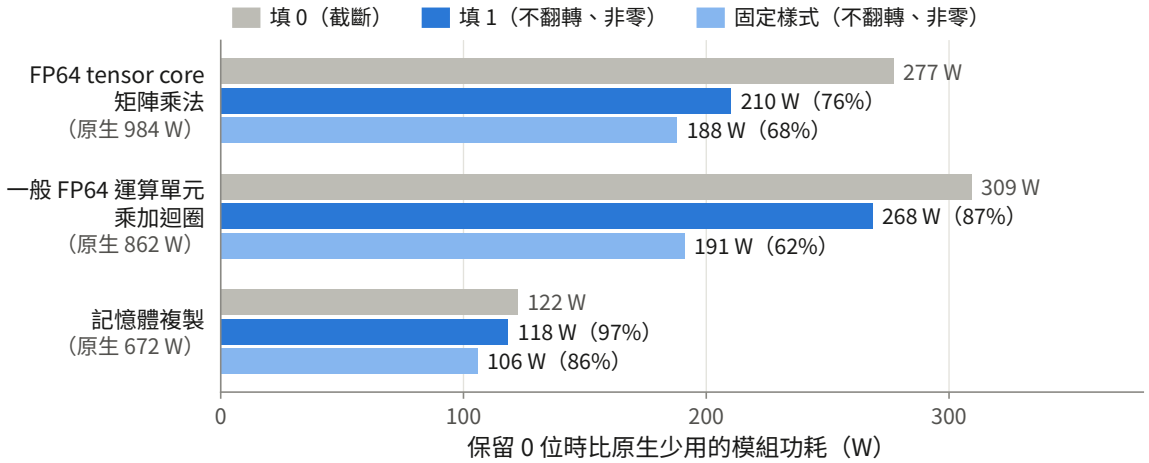


圖 5.1　省電機制：保留 0 位、被捨去的位元填 0、全 1 或固定樣式時，三個層次在 1000 W 下比原生少用的模組功耗。填 1 與固定樣式同樣不翻轉，但數值不為 0；括號內為佔填 0 省電的比例。

資料來源：Slurm 工作 1752–1753、1762、1815、1858；`gh200/results/m1_toggle_summary.csv` 等

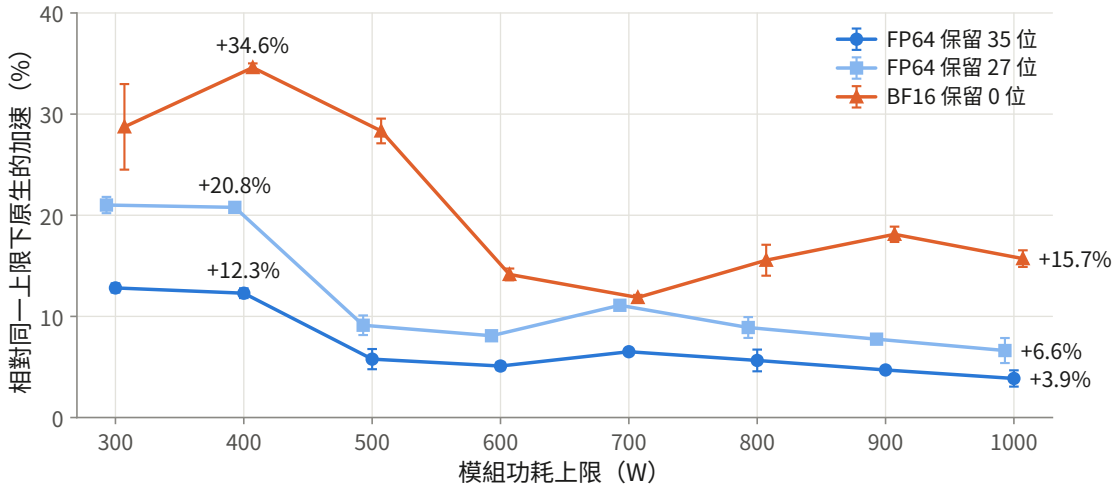


圖 5.2　FP64 與 BF16 矩陣乘法在各功耗上限下保留不同位數的加速（95% 信賴區間，3 個節點各 2 對；FP64 保留 27 位與 BF16 的點分別向左、右錯開約 7 W 以免重疊）。

資料來源：Slurm 工作 1667–1669（work1–3）；`gh200/results/e1_ci.csv`

（86–97%）；乘法器則還有額外的「數值為 0」效應：保留 20 位時，填 1 的低位與另一個運算元相乘的部分積仍會隨後者翻轉，填 0 則整片不動。保留 0 位時兩個運算元的尾數都是常數，部分積本來就不翻轉，但填 0 仍比填 1 省電（707 W 對 774 W），可能來自乘積的對齊與累加較單純（全 0 時乘積是 2 的冪），此點尚未驗證。在 500 W（功耗受限、比較效能；單一節點、各兩次）重做同樣的對照，「不翻轉」佔效益的 77–92%，結論相同。此結果在 GH200 的模組功耗上量化了前人提出的翻轉假說 [2]；資料搬移的部分也與 DRAM 能耗隨翻轉率變化的觀察一致 [15, 17]。

5.2　功耗上限下的矩陣乘法微基準

我們先在最單純的工作負載上量出效益的上界。測試使用 cuBLAS 矩陣乘法（FP64 為 $32768 \times 32768 \times 2048$，BF16 為 $32768 \times 32768 \times 6144$），輸入是 $[-1, 1)$ 的均勻隨機值；在 3 個節點、300–1000 W 共八個上限下，每個上限以交錯順序各執行 2 次（圖 5.2）。

大多數信賴區間只有 ± 0.1–1.2 個百分點，效益遠大於量測雜訊。換算成等效功耗，FP64 保留 27 位在 500–800 W 相當於多給 79–162 W，BF16 保留 0 位相當於多給 121 W 到 200 W 以上。最佳能效也隨之提高：FP64 從原生的 82 GFLOP/J（500 W）升到保留 27 位的 90 GFLOP/J（400 W），BF16 從 950 GFLOP/J（600 W）升到保留 0 位的 1159 GFLOP/J（500 W）。效益與功耗上限並非單調關係，在 300–400 W 最大。一個可能的解釋（尚未驗證）是：上限極低時電壓已在下限，功耗大致與時脈成正比，省下的功耗能換到較多時脈；而在 1000 W 時，FP64 與 BF16 矩陣乘法仍受功耗

上限限制，所以仍有 3.9%（FP64 保留 35 位）到 15.7%（BF16 保留 0 位）的加速。

需要強調的是，這些是均勻隨機資料、保留位數很低時的數字，屬於翻轉最劇烈的情況，代表效益的上界，而不是應用的操作點。下一節檢視具正確性檢查的完整程式。

5.3　HPL 與 HPL-MxP：具正確性檢查的完整基準

5.3.1　實驗設定與原生 HPL 的調校

HPL 的實驗使用其內建的量化選項：在每次 FP64 矩陣乘法之前，以一個捨入核心把兩個輸入捨入到 k 位（平手取偶的最近捨入），乘法本身仍是 FP64 cuBLAS。實驗使用 8 個節點、$N = 286720$（調參後的設定），在 400–1000 W 下每個上限以交錯順序各執行 2 次。

為了確保比較的基準是最佳的原生效能，我們先調校原生 HPL。最早的量測沿用 H200 時期的設定（$N = 262144$、$NB = 1024$、2×4 網格、$f = 0.05$、NCCL 預設值），1000 W 下原生只有 374.8–376.0 TFLOP/s。我們在 1000 W 掃描 N、NB、$P \times Q$、f 與 NCCL 參數（Slurm 工作 3303、3306，共 24 組設定、26 次執行）：NCCL 固定 8 個通道與 2 MiB 緩衝貢獻約 4.5%，N 加大到 286720 再多約 2%，4×2 網格反而慢 14%。同一個最佳設定重跑三次為 390.2–398.8 TFLOP/s（相差約 2%），f、NCCL 通道與緩衝、NB、N 的其他鄰近設定都沒有超出這個雜訊範圍（$f = 0.15$ 反而慢 3%）。以下的功耗上限掃描都使用這個設定（Slurm 工作 3315，work4–11）。

5.3.2　功耗上限下的效能

表 5.1 與圖 5.3 為調參後的結果。保留 36 位在 400–800 W 快 4.0–11.0%，換算成等效功耗多出 39–127 W（800 W 的 127 W 是在 800 與 1000 W 之間直線內插，偏向高估），每次求解的耗電量少 4.1–9.3%。在預設的 1000 W 上限下，調參後的原生 HPL 仍有 75% 的取樣撞上限（SM 平均 1936 MHz），截斷後只剩 7%（SM 1977 MHz），所以仍快 3.9%，模組功耗從 930 W 降到 886 W，每次求解少耗電 7.5%。未調參的設定（$N = 262144$，Slurm 工作 1670）在 400–800 W 快 3.4–8.3%、1000 W 快 0.7% 並省電 7.7%，結論相同。

依規格，每顆 GH200 的 FP64 tensor core 峰值為 67 TFLOP/s [41]，8 顆合計約 536 TFLOP/s。調參後的原生 HPL 在 1000 W 達到約 73%（重測兩次平均

表 5.1　調參後 8 節點 HPL 在各功耗上限下的效能（$N = 286720$，TFLOP/s，兩次的平均）

每節點模組上限	原生	保留 36 位	加速	等效多出的上限 ΔW	每次求解的耗電
400 W	225.0	249.9	+11.0%	39 W	−9.3%
500 W	288.0	300.7	+4.4%	42 W	−4.1%
600 W	318.0	334.0	+5.0%	57 W	−5.1%
700 W	346.2	362.9	+4.8%	68 W	−4.8%
800 W	370.8	385.6	+4.0%	127 W	−4.2%
1000 W	393.9	409.2	+3.9%	—	−7.5%

資料來源：Slurm 工作 3315；`gh200/results/e3_tuned_summary.csv`。截斷後的數字為研究用途，並非正式成績。

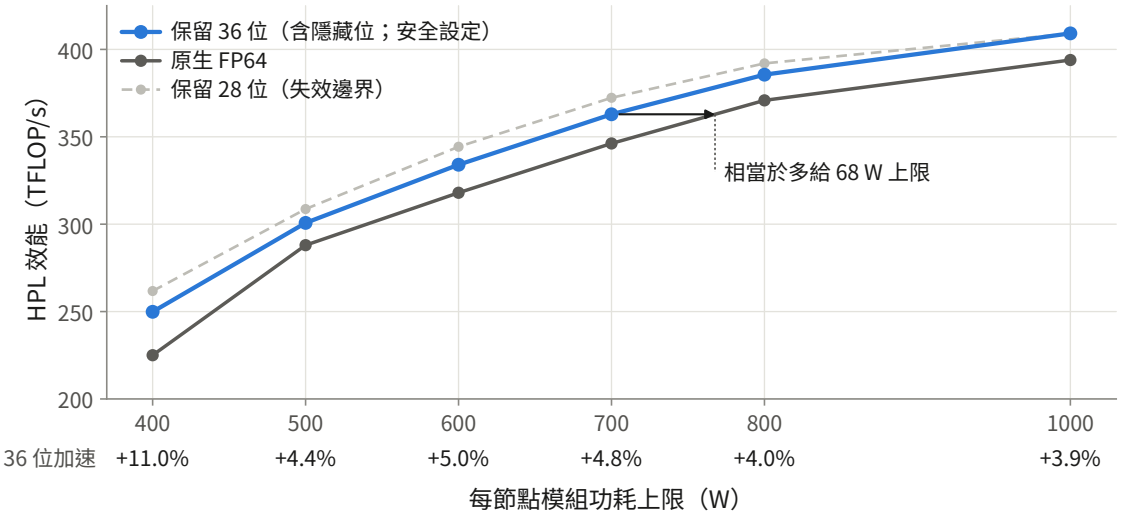


圖 5.3　調參後 8 節點 HPL 的效能與功耗上限：原生、保留 36 位與保留 28 位。保留 28 位的殘差已接近門檻，只作為失效邊界的參考。

資料來源：Slurm 工作 3315（調參後的功耗上限掃描）、1820（多組 N）；`gh200/results/e3_tuned_summary.csv`

393.9 TFLOP/s，調參時單次最高 398.8 TFLOP/s），保留 36 位達到 409.2 TFLOP/s（研究用途，並非正式成績）。功耗上限壓得越低，效能下降越多，截斷則把一部分損失的效能搶回來。保留 28 位可以再多 0–5 個百分點（例如 700 W 時 372.3 TFLOP/s），但它在這個 N 的殘差已達 15.4（門檻 16），在 $N = 196608$ 時更是失敗，所以不作為操作點。

5.3.3　殘差與失效邊界

HPL 最後會檢查縮放殘差是否小於 16。殘差會隨矩陣大小 N 變化，所以我們在 1000 W 以三組 N 檢查（每組 2 次，殘差完全重現），再加上調參後的 $N = 286720$（功耗上限掃描的 12 次，殘差也完全重現），結果列於表 5.2。

從 36 位減到 28 位少了 8 位，殘差放大 86–410 倍，量級與 $2^8 = 256$ 相符。這表

表 5.2　HPL 縮放殘差隨保留位數與矩陣大小 N 的變化（門檻 16）

保留位數	$N = 131072$	$N = 196608$	$N = 262144$	$N = 286720$	判定
原生（53 位）	1.5×10^{-4}	1.3×10^{-4}	2.6×10^{-5}	7.6×10^{-5}	通過
36	0.16	0.072	0.042	0.038	通過，距門檻約百倍（最小 98 倍）
28	13.97	**18.09**	14.70	15.40	$N = 196608$ 失敗，其餘都接近門檻
24	456	462	91	—	全部失敗

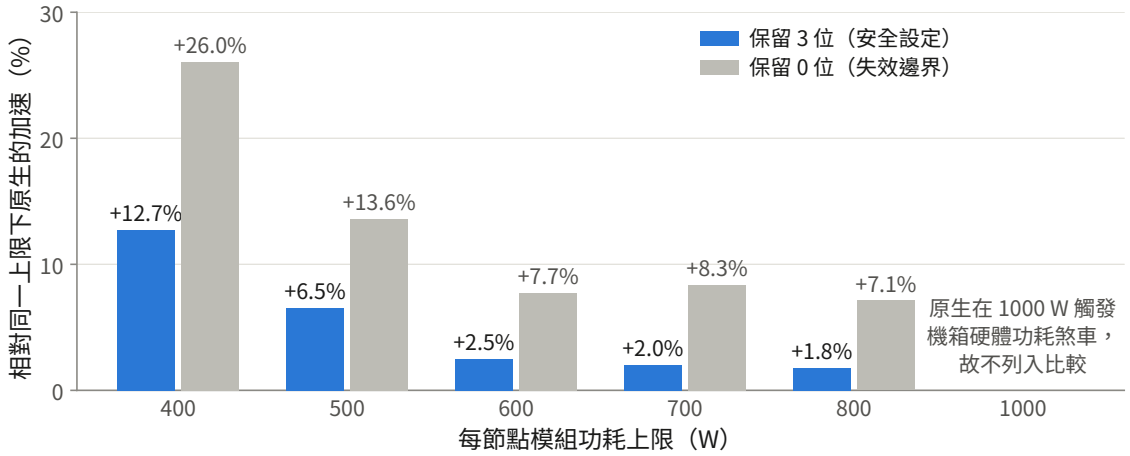


圖 5.4　8 節點 HPL-MxP 保留 3 位與 0 位在各功耗上限下的加速（對角優勢的測試矩陣，規則只允許用於測試）。

資料來源：Slurm 工作 1671；`gh200/results/e2_summary.csv`

示 28 位距門檻不到 1 位，換一個 N 就可能失敗；36 位則有約 7 位的餘裕，因此我們以 36 位作為 HPL 的操作點。

5.3.4　HPL-MxP

HPL-MxP 以 BF16 完成 LU 分解，再以 FP64 迭代修正。我們在每次寫回 BF16 時歸零，使用 8 個節點、$N = 491520$。此處使用的測試矩陣是對角優勢矩陣（對角元等於該列非對角元絕對值的和），對角 tile 使用近似分解（`--approx-diag`），迭代修正使用 Richardson 法搭配 FP32 預條件子。依 HPL-MxP 的規則，嚴格對角優勢的矩陣只能用於測試 [42]；在強對角優勢的矩陣上，很粗的 L、U 仍是好的預條件子，所以以下「保留 3 位仍穩定」的結論不能直接外推到正式允許的矩陣。

如圖 5.4 與表 5.3 所示，保留 3 位在 400–800 W 快 1.8–12.7%（等效多出 34–45 W、耗電量少 3.4–10.7%），在 $N = 245760$–491520 的四組大小都穩定：殘差 1.4–3.8，固定 3 次修正。每個上限的修正次數都從原生的 1 次變成 3 次；LU 本身的加速從 400 W 的 16.0% 降到 800 W 的 6.0%，扣除多出的修正時間後，600 W 以上只剩約

表 5.3　8 節點 HPL-MxP 在各功耗上限下的效能（$N = 491520$，TFLOP/s，報告值含迭代修正，兩次的平均）

每節點模組上限	原生 BF16	保留 3 位	加速	等效多出的上限 ΔW	每次求解的耗電
400 W	2076	2340	+12.7%	35 W	−10.7%
500 W	2823	3007	+6.5%	45 W	−6.6%
600 W	3229	3310	+2.5%	37 W	−3.7%
700 W	3449	3519	+2.0%	34 W	−4.7%
800 W	3655	3720	+1.8%	38 W	−3.4%
1000 W	3015*	3976	—	—	—

* 原生 HPL-MxP 在 1000 W 會觸發機箱的硬體功耗煞車，效能被拉低，不納入比較；等效上限改以同一組節點上另一個工作（1675）量測的 900 W 原生點（3830 TFLOP/s，兩次平均）換算。資料來源：`gh200/results/e2_summary.csv`。

2%。保留 0 位雖然快 7.1–26.0%，但 $N = 491520$ 的殘差已達 10.6–11.1（門檻 16），在 $N = 245760$ 時修正次數暴增到 52 次、效能下降 70%，所以只是失效邊界。

保留 0 位在這個 N 上更快（例如 400 W 時 2616 TFLOP/s），但在較小的 N 上修正次數暴增，所以不作為操作點。另有一個附帶發現：風扇全開時，原生 HPL-MxP 在 1000 W 會觸發機箱電源的硬體功耗煞車（每次 4–10 秒），反而比 800 W 慢；尾數歸零版本的尖峰功耗較低，在開啟計數器的診斷工作中，保留 0 位的版本沒有觸發。

5.3.5　小結

在具殘差檢查把關的完整基準上，功耗受限時尾數歸零都帶來正面收益；但保留位數必須以多組問題大小驗證，並選擇有餘裕的操作點；HPL-MxP 的結果目前限於對角優勢的測試矩陣。

5.4　科學軟體：Quantum ESPRESSO 與 PySCF

5.4.1　所需的尾數位數

我們以攔截函式庫截斷所有 FP64 矩陣乘法，觀察物理結果（總能量）在多少位時開始偏離，結果列於表 5.4。

平面波密度泛函只需要 20–28 位，高斯基底的量子化學需要 32–36 位；後者與 Dawson 等人以模擬得到的「約 37 位有效尾數即已足夠」一致 [40]。

表 5.4　科學軟體所需的尾數位數（攔截所有 FP64 矩陣乘法）

系統	結果
Quantum ESPRESSO 金表面（112 原子）	最近捨入 24 位誤差 8.1×10^{-6} Ry、20 位 5.5×10^{-5} Ry、16 位發散
Quantum ESPRESSO 矽（Si 1000）	28 位誤差 3.5×10^{-7} Ry；20 位是失效邊緣（一次收斂、一次中止）
PySCF 纈胺黴素 DFT（門檻 10^{-7}）	32–40 位都與原生同樣 12 圈收斂；32 位誤差 $1.2 \times 10^{-5}\ E_h$（約 0.007 kcal/mol）
PySCF 紫杉醇 HF+MP2	往零截斷 44 位誤差 $5.6 \times 10^{-8}\ E_h$；36 位 $2.7 \times 10^{-5}\ E_h$

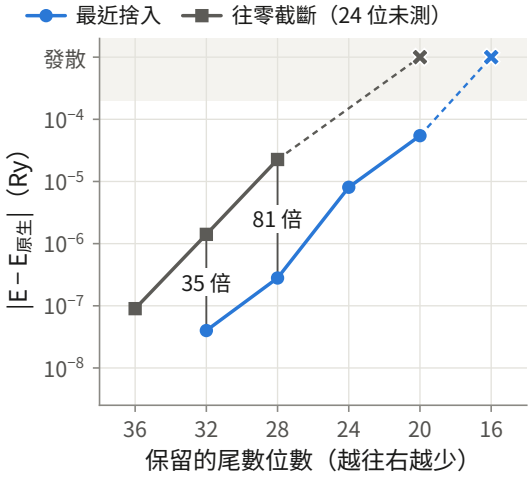


圖 5.5　Quantum ESPRESSO 金表面的總能量誤差與保留位數：往零截斷與最近捨入。
資料來源：Slurm 工作 1729、1738（金表面攔截掃描）

5.4.2　捨入方式的影響

如圖 5.5 所示，同樣保留 32 位與 28 位時，往零截斷的總能量誤差分別是最近捨入的 35 倍與 81 倍，遠大於單次矩陣乘法的差距（式 (4.3) 的係數 α 只差約 2.6 倍）：截斷永遠把數值往 0 拉，誤差有固定方向，會在自洽迭代中一路累積；最近捨入沒有偏差，誤差會互相抵銷。

5.4.3　效能與耗電

這些程式在 1000 W 下用不到上限，所以我們在會碰到上限的 450–550 W 量測，結果列於表 5.5。

每個工作的第一次原生執行偏慢（暖機，例如金表面 96.9 秒，其餘為 94.2–94.7 秒），含它的那一對收益偏大；排除這一對後，自動調整在金表面為 −3.9%、矽為 −2.3%。金表面上，自動調整在 6 對中每一對都比固定 24 位快 0.6–1.8 個百分點；矽上也每對都略快於固定 28 位，但只差 0.18 ± 0.14 個百分點。因此自動調整至少追平

表 5.5　Quantum ESPRESSO 程式內截斷與自動調整位元數的時間與耗電

做法	系統	上限	時間	耗電量	總能量誤差
依收斂門檻自動調整位元數	金表面	450 W	$-4.2 \pm 0.7\%$	$-3.9 \pm 1.1\%$	0（印出的每一位都相同）
依收斂門檻自動調整位元數	矽	550 W	$-2.4 \pm 0.5\%$	$-2.5 \pm 0.6\%$	0（印出的每一位都相同）
程式內固定 24 位	金表面	450 W	$-2.7 \pm 1.1\%$	$-2.7 \pm 1.6\%$	3×10^{-8} Ry
程式內固定 28 位	矽	550 W	$-2.3 \pm 0.6\%$	$-2.3 \pm 0.6\%$	$< 10^{-8}$ Ry
攔截所有 GEMM，最近捨入 28 位（函式庫內建時的上限）	矽	550 W	−6.2%	−6.4%	3.5×10^{-7} Ry

前四列為程式內的做法，與未修改的原生程式交錯執行 6 次（成對），數字為 6 對的平均 $\pm$ 95% t 區間；最後一列與同樣付出複製成本的基準比較，各執行一次。

事後才找得到的最佳固定位數，而且不需要事先試驗。最後一列攔截所有矩陣乘法，代表「如果函式庫內建截斷」時的上限；攔截本身約有 5% 的複製成本。

自動調整的規則（式 (4.1)）讓位元數隨收斂從 16 升到 39 位，使截斷誤差大致不超過 Davidson 當圈本來就容許的誤差，並由後續迭代修正；因此總能量在印出的所有位數（10^{-8} Ry）都與原生相同，SCF 圈數也相同。不過金表面的收益有一部分並非功耗效應：自動調整讓 Davidson 少做了約 7–9% 的工作（`h_psi` 呼叫 311 次對 336 次）。在一次功耗上限被外部重設為 1000 W、不受功耗限制的執行中（工作 2003，時脈相同），它仍比原生與固定 24 位快約 2%（73.5–73.7 秒對 74.8–75.1 秒），與 450 W 時領先固定 24 位的幅度相同。所以金表面 −4.2% 中約有一半來自迭代減少，原因尚未查明；矽上沒有這個現象（206 次對 205 次），−2.4% 可視為功耗效應。

5.4.4　單精度的可行性

Quantum ESPRESSO 的運算元值域放得進 FP32，所以我們也嘗試把所有 GEMM 改為 FP32、TF32 或 3xTF32（3xTF32 是攔截工具內自行撰寫的實作：拆成兩個 TF32、以三次 FP32 累加的 tensor core 乘法計算）。結果金表面的 FP32 在第 26 圈、TF32 與 3xTF32 在第 2 圈中止；矽的 FP32 在第 14 圈、3xTF32 在第 9 圈中止，積分電荷也明顯偏離。原因在第 5.5.4 節說明。這是把所有 GEMM 都替換掉的極端做法；只在部分步驟使用 FP32 的混合精度（例如 Woo 等人 [38]）不在此限。

5.4.5　小結

科學軟體需要 20–36 位，在功耗受限時能取得 2–6% 的時間與耗電改善。最有價值的是「隨收斂門檻調整位元數」：總能量印出的每一位都不變，而且不需要事先知道安全的位數。

5.5　與 FP64 模擬及單精度的分界

以較少位元計算 FP64 矩陣乘法還有另一條路：以 INT8 tensor core 模擬 FP64。我們在同一節點、同一功耗上限下，把誤差對齊到同一量級後比較速度。以小矩陣與 CPU 精確值對照確認：在相同位元數下，截斷與模擬的誤差屬於同一量級。

5.5.1　功耗上限改變勝負

圖 5.6 比較「方陣」（$m = n = 32768$、$k = 2048$ 的合成資料，以下沿用此稱呼）上的各種做法。GEMMul8 的功耗很高：1000 W 時就用到約 970 W，450 W 時時脈掉到 650–720 MHz，反而輸給截斷。cuBLAS 模擬在 1000 W 只用約 706 W；相反地，cuBLAS 模擬在方陣上隨上限降低而更有優勢（28 位時從 1000 W 的 1.07 倍升到 450 W 的 1.20 倍）。所以上限收緊對誰有利，取決於模擬本身的功耗；截斷在每個上限都比原生快，但幅度不隨上限單調變化（第 5.2 節）。在較粗的精度等級（10^{-9}）則是模擬較快；cuBLAS 模擬以約 8 位為一片，精度呈階梯式變化，截斷則能以 1 位為單位調整。

5.5.2　矩陣形狀決定勝負

我們攔截 PySCF 纈胺黴素計算的全部 45,574 次 GEMM，發現兩種形狀各佔 35% 的運算量（圖 5.7）。模擬需要先把輸入拆片、最後再合併，這部分的成本與矩陣元素數（約 $mk + kn + mn$）成正比，而乘法本身是 mnk。在 $n = 300$ 的瘦長矩陣上，拆片成本與乘法相當：與截斷 36 位同精度的 cuBLAS 模擬（32 位）在 1000 ／ 575 W 只有原生 0.38 ／ 0.45 倍的速度，較粗的 28 位也只有 0.47 ／ 0.57 倍；在 $k = 76800$ 的長內積矩陣上，拆片成本被均攤，INT8 的高運算量便顯現出來。截斷沒有拆片成本，在同一個上限下對任何形狀的加速都差不多（例如 1000 W 時兩種形狀都快約 5–6%）。因此最佳做法是依形狀選擇：瘦長矩陣用截斷，長內積矩陣用模擬。

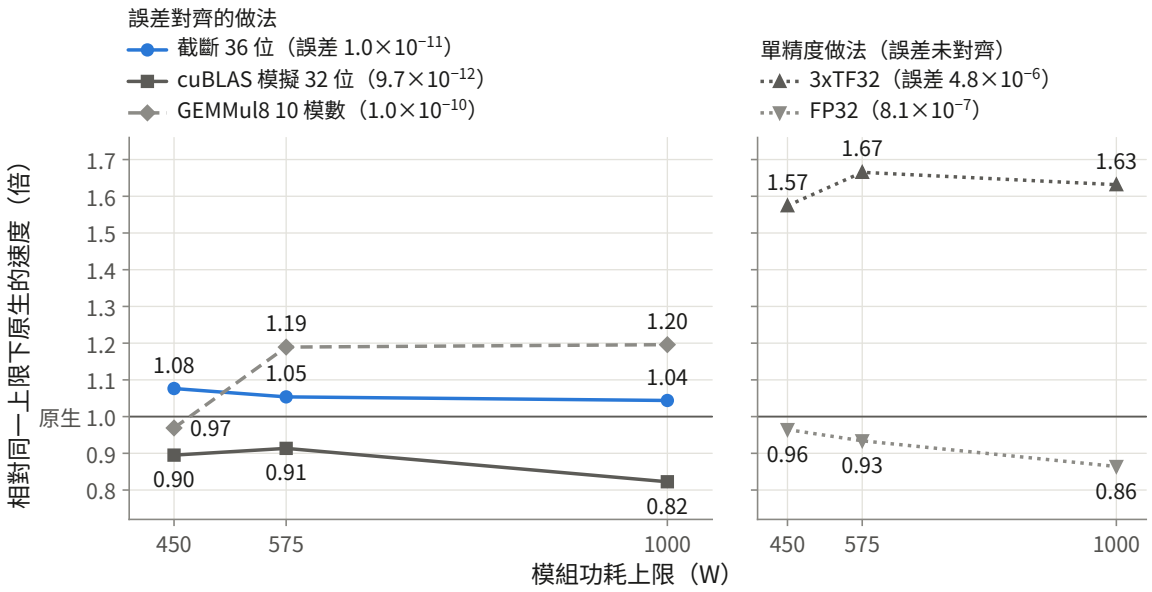


圖 5.6　方陣 FP64 矩陣乘法在 450、575 與 1000 W 下相對原生的速度。左：誤差對齊的截斷 36 位與 INT8 模擬（cuBLAS 32 位、GEMMul8 10 模數）；右：單精度做法（3xTF32、FP32，誤差未對齊）。

資料來源：Slurm 工作 1894；`gh200/results/m3_ozaki2_tf32x3_summary.txt`

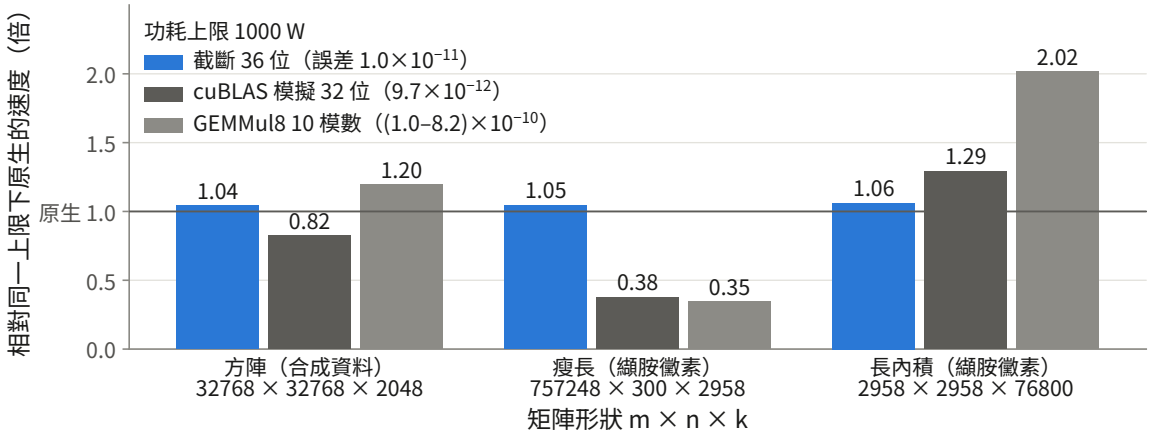


圖 5.7　1000 W 下，繆胺徽素 DFT 的兩種真實矩陣形狀（瘦長 757248 × 300 × 2958 與長內積 2958 × 2958 × 76800）及作為參考的方陣上，截斷與模擬相對原生的速度。

資料來源：Slurm 工作 1894、1896、1897；`gh200/results/m3_realshape_summary.txt`

5.5.3　分界點掃描與預測公式

為了把「依形狀選擇」變成可用的規則，我們固定 $m = 8192$，n 取 256、1024、4096、8192，k 取 256 到 81920，在 1000、575 與 450 W 各量測兩輪，共 480 筆（圖 5.8）。

截斷保留 36 位在全部 60 組（形狀 × 上限）都比原生快（450 W 為 3.8–11.9%、575 W 為 3.6–5.9%、1000 W 為 0.5–4.8%）。模擬只在 n 與 k 都大時較快：cuBLAS 模擬需要 $n \geq 4096$ 且 $k \geq 16384$，勝出的形狀在 1000、575、450 W 分別有 5、5、2 種；

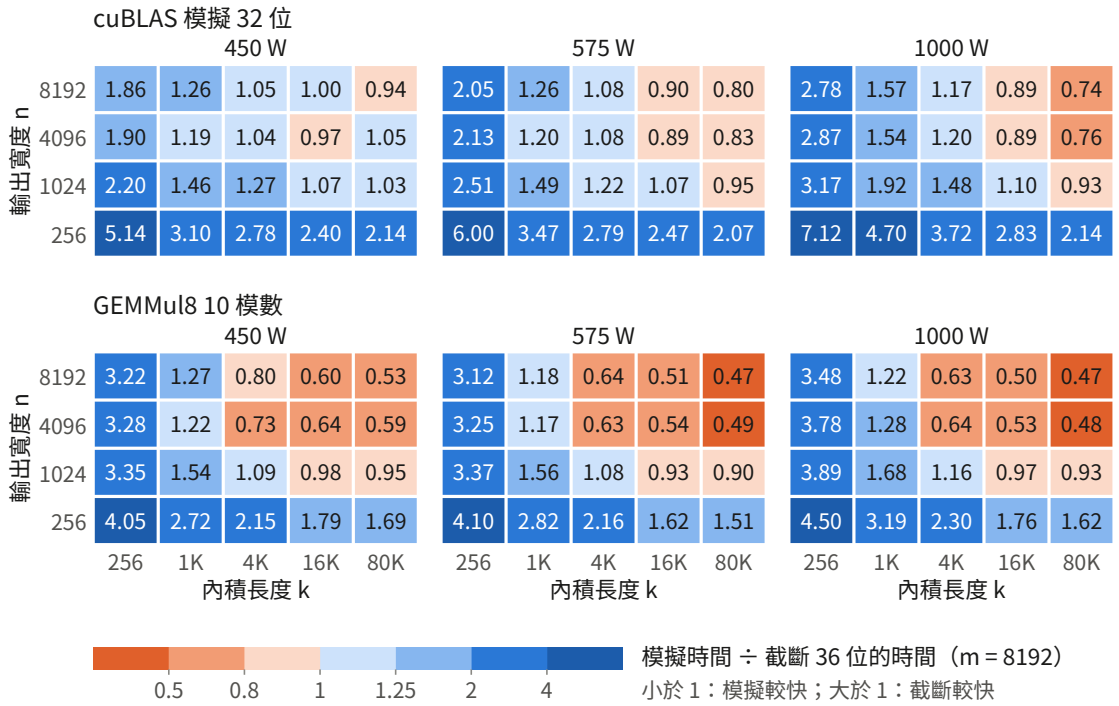


圖 5.8　截斷與 INT8 模擬的分界點掃描（$m = 8192$）：模擬時間相對截斷保留 36 位的倍數。
資料來源：Slurm 工作 2005–2008（480 筆）；`gh200/results/crossover_*.csv`

但在模擬本身用不滿功耗的形狀上（例如 $n = 8192$、$k = 4096$，模擬對截斷的時間比從 1.17 降到 1.05），上限越低反而對模擬越有利，方向取決於模擬本身是否受功耗限制。GEMMul8 需要 $n \geq 4096$ 且 $k \geq 4096$，在那裡時間只要截斷的 0.47–0.80 倍，但精度低一個量級（10^{-10} 對 10^{-11}）。以

$$\frac{t_{\text{emu}}}{t_{\text{trunc}}} = A + B\left(\frac{1}{m} + \frac{1}{n}\right) + \frac{C}{k} \tag{5.1}$$

擬合，GEMMul8 的決定係數 R^2 達 0.98–0.99；若只用單一的有效大小 $1/(1/m + 1/n + 1/k)$ 擬合，則只有 0.86–0.89，表示 GEMMul8 的輸入拆片與輸出合併是兩種不同的成本。cuBLAS 模擬的 R^2 只有 0.88–0.89，與單一有效大小的模型相近，仍有模型未捕捉的因素（例如不同形狀選到不同核心）。四個 n 值分別在不同節點上量測；每格的比值是同節點配對，但跨形狀、跨上限的趨勢可能混入節點差異。

式 (5.1) 只以 $m = 8192$ 的資料擬合。我們以先前量測、未參與擬合的兩種纈胺黴素真實形狀檢驗（1000 W 與 575 W、兩種模擬方法，共 8 個案例）：瘦長的 $757248 \times 300 \times 2958$ 預測截斷勝出、長內積的 $2958 \times 2958 \times 76800$ 預測模擬勝出，8 個案例的勝負全部判斷正確。倍數本身的誤差在 cuBLAS 模擬為 6–15%、GEMMul8 為 24–37%，後者較大是因為瘦長矩陣的 $m = 757248$ 遠超出擬合範圍。對使用者而言，判斷該用哪一種只需要勝負正確，這個精度已經足夠。不過這 8 個案例都離分界很遠（模擬的時間不是截斷的一半到八成，就是兩倍以上），只是寬鬆的檢驗；公

表 5.6　單精度做法的矩陣乘法相對誤差隨內積長度 k 的變化

內積長度 k	3xTF32	TF32	FP32
2048	4.8×10^{-6}	2.6×10^{-4}	8.1×10^{-7}
16384	3.8×10^{-5}	—	2.3×10^{-6}
95386（金表面的平面波數）	5.6×10^{-5}	2.7×10^{-4}	2.8×10^{-6}
375286（矽的平面波數）	2.2×10^{-4}	3.4×10^{-4}	1.9×10^{-6}

式對 m 的依賴也尚未驗證。

5.5.4　FP32 與 3xTF32 不適用的原因

3xTF32（自行撰寫的實作，不是 CUTLASS 的版本）在方陣上最快，但它的誤差隨內積長度 k 成長；截斷後的 FP64 仍以 FP64 累加，誤差與 k 無關（表 5.6）。

在 4096×4096 上，k 每增加 8 倍，3xTF32 的誤差就增加 7.5–7.9 倍，與 tensor core 以 FP32 累加時朝零捨入的既有觀察一致 [43]。Quantum ESPRESSO 的 GEMM 內積長度就是平面波數（數十萬），所以 3xTF32 退化到與 TF32 同一量級，實測無法收斂（第 5.4.4 節）。

5.5.5　小結

截斷與模擬沒有一方全面勝出，分界點由所需精度、矩陣形狀與功耗上限決定，兩者應依形狀互補使用。把所有 GEMM 換成單精度（FP32、3xTF32），在長內積上無法取代 FP64 截斷。

5.6　非矩陣乘法的 FP64 程式

5.6.1　核心層級

Ozaki 類模擬只適用於稠密矩陣乘法，因此若尾數歸零在其他 FP64 運算上也有效，它便有模擬無法取代的空間。我們先在核心層級測試：只用一般 FP64 運算單元的乘加迴圈、記憶體複製與 triad、N 體重力計算，以及一個 CG 稀疏求解器（圖 5.9）。

在低功耗上限下，這些省電會轉成加速：乘加迴圈在 500 W 快 4.8–9.0%（36–20 位），記憶體複製在 450 W 快 7.7–14.0%。但 N 體計算只快 1–4%：它第一步就是座標相減 $dx = x_j - x_i$，差值的尾數又是滿的，後面約 20 個運算照常翻轉。這說明效益

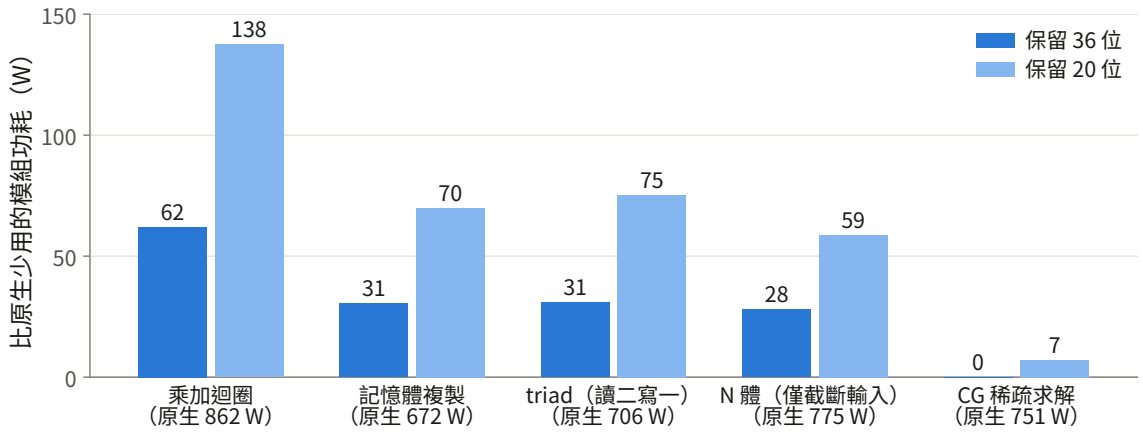


圖 5.9　非矩陣乘法核心在 1000 W 下保留 36 位與 20 位時，比原生減少的模組功耗。
資料來源：Slurm 工作 1815、1858、1860；`gh200/results/m4_*_summary.txt`

表 5.7　第三方非矩陣乘法程式的係數截斷結果

軟體	被截斷的係數	係數佔記憶體流量	結果
MFEM（免組裝有限元素，3827 萬未知數）	每個求積點的幾何與係數資料	約 43%	450 W 快 0.9%（36 位）／2.0%（20 位），550 W 快 0.7%／1.4%；解的相對變化 $\le 2.3 \times 10^{-11}$
SW4（地震波，1.26 億格點）	材料參數 ρ、μ、λ	約 9%	350 W 快 0.6%（36 位）／1.3%（20 位），450 W 快 0.3%／0.5%；波形最大相對差 $\le 3.4 \times 10^{-6}$
LAMMPS（EAM 分子動力學）	位能表	約 0%（常駐快取）	各上限 0.0–0.4%，落在雜訊內（負面對照成立）

隨「計算深度」遞減，只有直接讀取截斷資料的運算受惠。CG 幾乎沒有效益，因為被截斷的矩陣值只佔每圈記憶體流量的一部分，索引與向量不能截斷。

5.6.2　第三方真實軟體

為了確認核心層級的結果能否延伸到真實程式，我們在三個不以矩陣乘法為主的 FP64 GPU 程式中，於設定階段對「儲存後每一步都會被乘進去的係數陣列」歸零一次（不設定時與原版逐位元相同），結果列於表 5.7。

MFEM 的係數佔了近一半的記憶體流量、直接進入乘法器，收益仍只有 1–2%。三者的收益順序（MFEM ≥ SW4 > LAMMPS）與係數佔記憶體流量的比例一致。

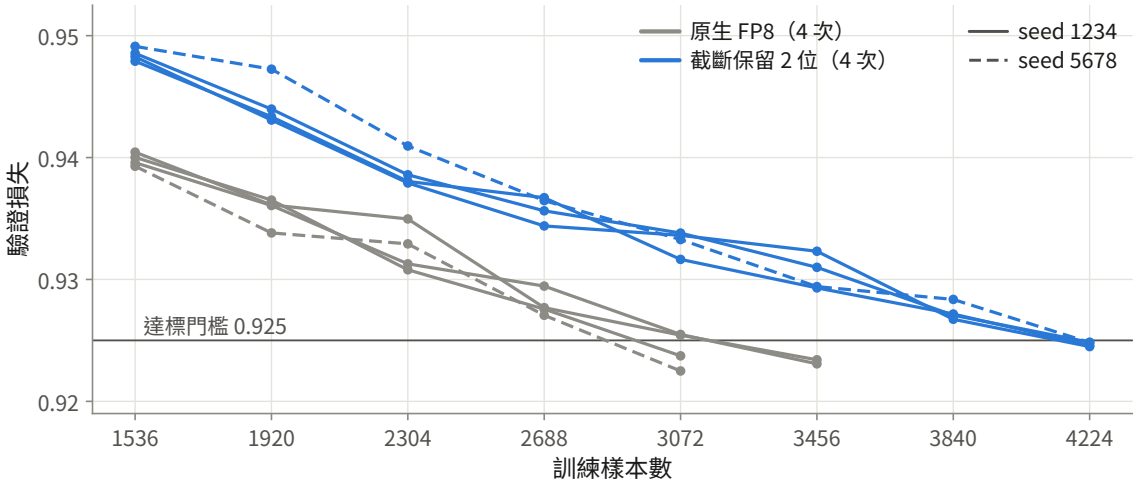


圖 5.10　Llama 2 70B LoRA 原生 FP8 與截斷保留 2 位的驗證損失隨訓練樣本數的變化（實線為 seed 1234、虛線為 seed 5678；水平線為達標門檻 0.925）。

資料來源：Slurm 工作 1732、1736、1760 起、1901、1904、1990、1993（分層截斷 2011、2020、2027 只列於內文）；`gh200/logs/lora`

5.6.3　小結

尾數歸零的省電不限於 tensor core，一般 FP64 運算單元與記憶體搬移也能省電；但在真實程式中，被截斷的資料只是耗電的一部分，核心層級的 7–18% 會被其餘滿尾數的運算稀釋到 1–2%。

5.7　AI 工作負載

5.7.1　訓練：Llama 2 70B LoRA 微調

MLPerf Training 比較的是「達到目標品質所需的時間」（驗證損失 ≤ 0.925）。基底權重已經是 FP8（只有 3 個尾數位元），我們在載入時再截斷一次，使用 8 個節點（圖 5.10）。

兩個亂數種子、四組配對（575 W 一組、800 W 三組）的結果一致：截斷到 2 位的驗證損失從第一次評估起就高 0.008–0.010，需要多花 2–3 個評估區間才達標（例如 seed 5678：原生 3072 個樣本、22.74 分鐘；截斷 4224 個樣本、31.56 分鐘），而每一步只快約 1%。截斷在載入時就削弱了基底模型，LoRA 無法補回。最近捨入與隨機捨入也沒有改善（分別需要 5376 與 4224 個樣本）。只截斷部分層的結果相同：只截斷 MLP 需要 4224 個樣本，與全部截斷相同；只截斷注意力需要 3840 個，退化落在雜訊內，但注意力只佔線性層權重約 18%，可以節省的部分本來就很少。只截斷注意力的設定以完全相同的參數再跑一次，第一次評估幾乎相同（0.9428 對 0.9425），

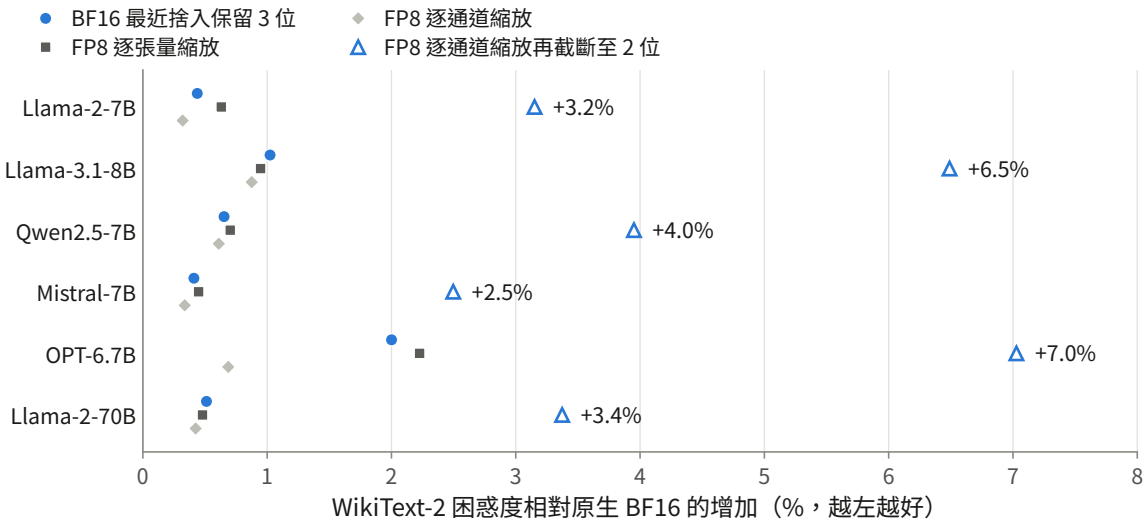


圖 5.11　六個語言模型在各格式下的 WikiText-2 困惑度相對原生 BF16 的增加（越左越好）。資料來源：`gh200/results/infer_ppl.csv`（70B 為逐層評估）

表 5.8　BF16 最近捨入保留 3 位與 FP8 在 FP8 容易出問題的場景下的比較

場景	BF16 最近捨入保留 3 位	FP8
注意力（2k ／ 8k 上下文的困惑度增加）	+0.49% ／ +1.06%	+0.52% ／ +1.08%
長上下文鑰匙檢索（8k–32k）	100%	100%
BERT-large SQuAD（MLPerf 99.9% 門檻）	通過	通過
CogVideoX-2B 影片（相對原生的 PSNR）	20–29 dB	18–38 dB

達標點卻差一個區間（3840 對 4224），所以我們以第一次評估的差距作為主要證據。結論是：對已經使用 FP8 的訓練，尾數已經沒有可以再節省的空間。

5.7.2　推論：BF16 截斷與 FP8 的比較

我們尋找兩種可能的機會：比 FP8 更快、精度又不會差太多的做法；或 BF16 加上尾數截斷比 FP8 更好的場景（圖 5.11）。

以最近捨入保留 3 位的 BF16 與 FP8 的尾數位數相同，矩陣乘法誤差也相當（均勻資料上 3.3% 對 3.6–3.9%；截斷需保留 4 位才接近，約 4.4%）。我們接著在文獻中 FP8 最容易出問題的場景比較兩者，結果都打平（表 5.8）。

另一方面，FP8 的矩陣乘法快 1.7–1.9 倍（575 W 實測）。在不改用結構化稀疏或 INT4 權重的前提下，更快的路只剩在 FP8 之上再截斷：以 Llama2-70B 的真實權重，兩邊都截斷到 2 位快 3.2–8.2%，但困惑度增加 3.4%，速度與精度大致以 1 比 1 交換；decode 階段的省電也只有 1.5–5.5%。

表 5.9　DF-MP2（紫杉醇，def2-TZVP）各變體的 MP2 時間與相對原生的加速

	原生	截斷 32 位	截斷 20 位	內建 FP32	cuBLAS 模擬 32 位
能量誤差	—	$10^{-11}\,E_\mathrm{h}$	$10^{-8}\,E_\mathrm{h}$	$(3\text{–}4)\times10^{-8}\,E_\mathrm{h}$	$2\times10^{-8}\,E_\mathrm{h}$
1000 W	39.5 s	+0.3%	+0.8%	+2.6%、+11.5%*	慢 36%
700 W	44.1 s	+5.3%	+7.6%	+18%	慢 28%
500 W	52.0 s	+4.9%	+8.3%	+15%	慢 18%
1000 W 時少用的功耗（耗電）	—	61–70 W（8%）	106–115 W（14%）	—	—

原生欄為 MP2 階段的時間，其餘為相對原生的加速；最後一列為 1000 W 時比原生少用的模組功耗與每次計算少耗的電。* 兩輪的結果，差距大。

5.7.3　小結

在 Hopper 上，FP8 已經取得「減少尾數位元」幾乎全部的好處。尾數歸零的價值在 FP64 科學計算，而不在已經使用 FP8 的 AI。

5.8　高功耗的真實應用：DF-MP2

第 5.4 節的科學軟體在 1000 W 下平均用不到上限（金表面約 535–600 W、矽 1000 約 800 W），必須壓低功耗上限才看得到明顯的加速。因此我們另外選擇一個高功耗的應用：GPU4PySCF 的密度擬合二階微擾理論（DF-MP2）。它的 MP2 階段 GPU 功耗約 800 W（1000 W 時模組約 870 W，只有部分時間撞上限），矩陣乘法佔 GPU 時間的 78%，而且幾乎都是同一種收縮：把密度擬合張量的兩個切片相乘（FP64 DGEMM）。截斷實作於程式內：張量建好後原地截斷一次尾數，之後所有矩陣乘法都受惠，不需要複製。同一個程序先做一次 HF，再依序執行各種 MP2 版本（紫杉醇，def2-TZVP 基底，每個上限交錯執行兩輪，截斷變體的兩輪差距都小於 0.6 個百分點），結果列於表 5.9。

截斷在功耗受限時快 4.9–8.3%；1000 W 時速度幾乎不變（+0.3–0.8%），但少用 61–115 W，與 HPL 同一量級。GPU4PySCF 內建的 FP32 更快，誤差也在化學精度內，因為這個張量的指數跨度很窄（整體 27 個、列內中位數 18 個 2 的次方），FP32 放得下。cuBLAS 模擬在這個形狀一律較慢。

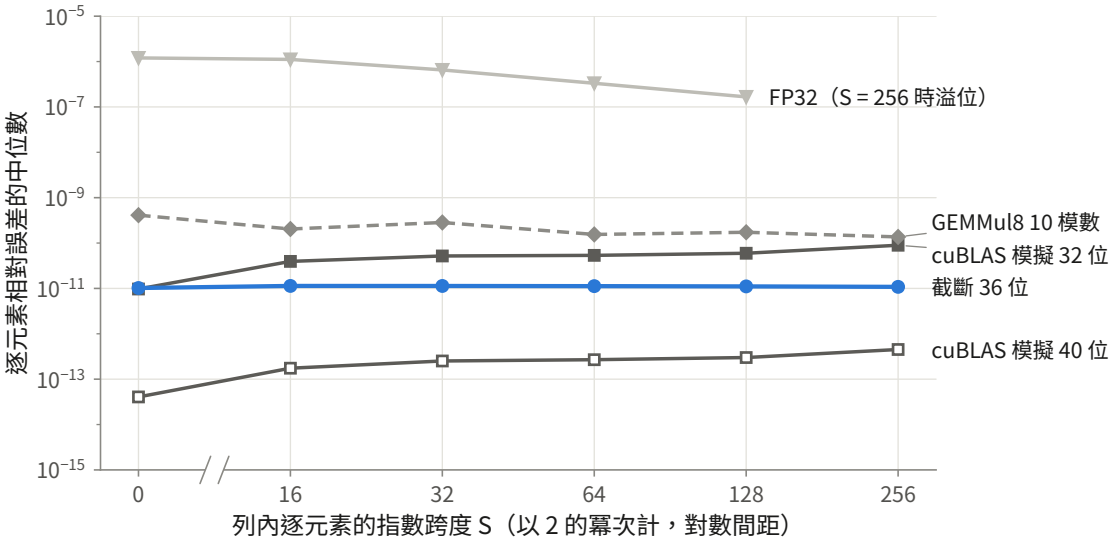


圖 5.12　指數跨度 S 變大時，各方法逐元素相對誤差的中位數。
資料來源：Slurm 工作 2034–2036（1000 ／ 575 W，兩輪）；`gh200/results/range_sweep.csv`

小結　在高功耗的真實應用上，截斷取得與 HPL 同一量級的收益；但資料跨度窄時，改變格式（FP32）可能更快。下一節說明改變格式在什麼情況下會出問題。

5.9　指數跨度：只能以截斷處理的資料

截斷保留每個元素自己的 11 位指數，只改變尾數；改變格式的做法都必須先處理指數：FP32 與 FP8 以整個張量的範圍為準，INT8 模擬（Ozaki 類）以每一列的最大值為基準切片，比最大值小 2^{-k} 以上的元素在保留 k 位時會被捨去。因此「數值大小相差多少」會限制整個矩陣共用縮放的換格式做法，而截斷不受影響。

5.9.1　合成實驗

我們讓矩陣乘法（8192 × 8192 × 16384；此形狀在跨度小時是模擬較快）的每個元素，其指數在 S 個 2 的次方內均勻分佈（圖 5.12）。

截斷 36 位的誤差與加速（575 W 快 5.7–6.3%、1000 W 快 4.4–4.5%）完全不受跨度影響。固定位數的 cuBLAS 模擬精度變差 4–9 倍（相當於少了 2–3 位），而模擬以約 8 位為一片，必須多付一整片才能追上；保證 FP64 精度的動態模式所需位數從 58 升到 301 位，超過上限就改用原生 FP64，連同判斷跨度的額外工作，時間是截斷的 1.6–2.6 倍；FP32 在跨度 256 時溢位。GEMMul8 在這種均勻分佈的資料上幾乎不受影響。另外，若只是每列各自有一個比例（整體跨度寬、列內窄），模擬完全不受影響，但 FP32 仍然溢位。

表 5.10　各應用的列內指數跨度，以及截斷與 INT8 模擬的誤差

應用	列內比最大值小 2^{-32} 以上的元素	截斷 36 位	cuBLAS 模擬固定 36 位（5 片）	GEMMul8（10 模數）
PySCF 纈胺徽素 DFT	24–44%（格點上的原子軌域矩陣達 87%）	$9.6 \times 10^{-7}\ E_h$	**0.0525 E_h（約 33 kcal/mol）**	發散
QE 矽	0.24–0.54%	0 Ry	6×10^{-8} Ry	記憶體不足
QE 金表面	0.008–0.02%	0 Ry	5×10^{-8} Ry	3.7×10^{-7} Ry

元素比例依運算量加權。QE 金表面的跨度統計取自只跑 2 圈的基準輸入，能量誤差取自完整收斂的輸入。「記憶體不足」指 GPU 記憶體不足。

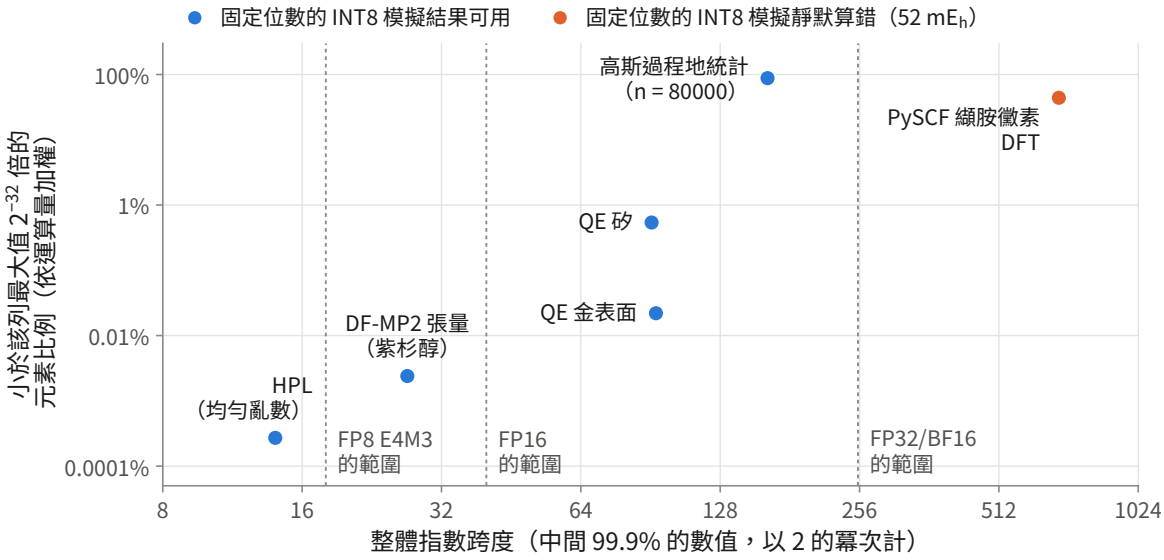


圖 5.13　各應用的整體指數跨度與列內小元素比例，以及固定位數的 INT8 模擬是否算錯。資料來源：`gh200/results/range_map_points.csv`（每個點的來源列於檔內；HPL 與高斯過程的點來自工作 3291、3304、3305）

5.9.2　真實應用

我們在攔截工具中加入逐列跨度統計（每個矩陣乘法抽樣 256 列／行，並以合成資料驗證與理論值相符），量出各應用中「比所在列最大值小 2^{-32} 以上」的元素比例，並在各應用上直接比較截斷與模擬（表 5.10、圖 5.13）。

同樣是模擬 36 位，在列內跨度窄的 QE 上完全正常；在列內跨度寬的 DFT 上，SCF 照常收斂、沒有任何警告，能量卻錯了 52 mE_h，而截斷在兩者上都正常。模擬加到 40 位（6 片）就能算對（誤差 $8 \times 10^{-8}\ E_h$），但時間是截斷 36 位的約 1.5 倍（第 5.10 節）；出錯的是使用者指定的固定位數模式。cuBLAS 的自動模式（依指數跨度決定位數，上限設為 79 位）則沒有算錯（誤差 $4 \times 10^{-9}\ E_h$）：89% 的運算量需要 80 位以上而改用原生 FP64，其餘以 72–79 位模擬，加上判斷跨度的額外工作，時間是原生的 2.1 倍（1000 W；Slurm 工作 3292）。所以寬跨度資料上的模擬並非不能用，而是不划算。反過來，跨度很窄的 HPL（依運算量加權，只有 2.7×10^{-6} 的元素比

所在列最大值小 2^{-32} 以上）以固定 36 位模擬也通過殘差檢查（0.197；截斷 36 位為 0.057），但在單 GPU、$N = 81920$ 時只有原生 65% 的速度（工作 3304、3305）。合成的均勻分佈低估了這個問題，因為真實的原子軌域值隨距離指數衰減，同一列中很多元素比最大值小得多。

5.9.3　小結

指數跨度會限制改變格式的做法：逐張量縮放的格式取決於整體跨度，逐列切片的 INT8 模擬取決於列內跨度。跨度寬時，整個矩陣共用縮放的格式或固定位數的模擬可能出錯，截斷則不受影響；但列內跨度寬只是模擬出錯的必要條件（第 5.10 節的高斯過程就沒有出錯），以 tile 為單位改變格式也能處理這類資料。列內跨度可以用攔截工具事先量測，作為選擇方法時的風險指標。

5.10　跨度寬且功耗高的應用：高斯過程地統計

前兩節的應用不是跨度寬，就是功耗高，沒有兩者兼具：幾種 DFT 變體的跨度都寬，但在 1000 W 下只用到 650–690 W。不過只要功耗上限低於它們的用電，寬跨度的 DFT 也會受限：纈胺黴素（格點等級 6）在 550 ／ 450 W 時，截斷 36 位快 3.3–3.5%、省電 2–3%，誤差 $10^{-6}\,E_{\mathrm{h}}$；模擬要用到 40 位才不會算錯，時間是截斷的 1.5 倍。

5.10.1　對數概似評估

為了找到兩者兼具的例子，我們轉向地統計的高斯過程最大概似估計（ExaGeoStat 這類應用）。每次評估對數概似都要對 $n \times n$ 的 Matérn 共變異矩陣做 Cholesky 分解，計算量與 HPL 一樣幾乎全是矩陣乘法；共變異隨距離指數衰減，所以同一列中數值大小相差很多。我們撰寫了一個 ExaGeoStat 式的核心（$n = 80000$ 個隨機位置、分塊 Cholesky、尾端更新為 FP64 矩陣乘法），在 6 個範圍參數上評估對數概似；截斷實作於程式內（每個面板在尾端更新前，以依記憶體順序走訪的單次融合核心截斷一次；早先兩次分開、存取不合併的版本只快 5.2–5.7%）。原生在 1000 W 用到 953–970 W，面板有 88% 的元素比所在列最大值小 2^{-32} 以上，跨度寬與功耗高兩者兼具。結果列於表 5.11。

截斷 28 位在功耗受限時快 7.4–8.2%；1000 W 時這組也快 3.0–3.9% 並省電 7–9%，但 1000 W 的加速隨設定而變（第 5.10.3 節依 Morton 排序的 $n = 60000$ 只快

表 5.11　高斯過程對數概似評估相對原生 FP64 的加速與偏差（$n = 80000$，6 個範圍參數，兩輪）

上限	截斷 36 位（偏差 2×10^{-5}）	截斷 28 位（7×10^{-4}）	cuBLAS 模擬 36 位（2×10^{-3}）	FP32（87）
1000 W	+2.4% ／ +3.2%，省電 2.6–4.5%	+3.0% ／ +3.9%，省電 7.2–8.7%	慢 41%	慢 8.9% ／ 8.2%
700 W	+4.9% ／ +5.1%	+8.2% ／ +7.6%	慢 31%	+3.5% ／ +3.0%
500 W	+5.0% ／ +4.7%	+7.7% ／ +7.4%	慢 21%	+4.3% ／ +3.6%

偏差為對數概似的最大絕對偏差（對數概似的數值約為 10^5）；「／」分隔兩輪的結果。

0.0–0.1%，因為原生只有部分時間撞上限）；偏差對參數估計沒有影響。FP32 的偏差大到無法用於估計（$n = 8000$ 時 FP32 仍估出相同的值，n 變大後才偏離，可能與條件數隨 n 變差有關）；cuBLAS 模擬比截斷 28 位不準，又慢 21–41%（尾端更新的 k 等於區塊大小 1024，依第 5.5 節的分界，這個形狀本來就不利於模擬）。另外，這裡的列內跨度比 DFT 更寬，模擬卻沒有算錯，只是精度變差：列內跨度寬是模擬出錯的必要條件，而非充分條件。

5.10.2　最大概似估計

表 5.11 是在固定的 6 個範圍參數上評估；為了確認估出的參數本身不變，我們也執行一維的估計：變異數與平滑度固定，以黃金分割搜尋範圍參數（取對數），每個變體評估 17 次對數概似（$n = 60000$，1000 與 700 W，各交錯兩輪；Slurm 工作 3249；`gh200/results/gp_mle_summary.csv`）。截斷 36 位、28 位與 cuBLAS 模擬估出的範圍參數都與原生完全相同（$\hat{\theta} = 0.05007952$，印出的 8 位全同），最大對數概似只差 4.8×10^{-6} 與 2.3×10^{-4}；FP32 的估計偏了 0.49%，對數概似低 0.72（$n = 8000$ 時連 FP32 都估出同一個值）。700 W 時整個估計以截斷 28 位快 7.1–7.3%、36 位快 4.8–4.9%，耗電少 6.6–6.8% 與 4.5–4.8%；1000 W 時也快 1.8–2.3% 與 1.6–2.2%，耗電少 9.1–9.9% 與 3.5–5.6%；cuBLAS 模擬慢 32–41%。

5.10.3　與 tile 混合精度疊加

前人的地統計混合精度（第 3.5 節）是把位置依空間填充曲線排序後，把整個 tile 換成低精度格式。我們在自寫的核心中加入類似的變體：位置依 Morton 曲線排序；在尾端更新中，每個 tile 欄在 FP64 範圍以外的 tile（依 tile 範數規則決定，要求精度 10^{-9}，同文獻 [36]）改用 FP16 輸入、FP32 累加的 tensor core。範圍參數 0.02 時有 52% 的 tile 使用 FP16，0.08 時只有 3%。六個範圍參數合計的結果列於表 5.12

表 5.12　截斷與 tile 混合精度疊加的效果（n = 60000，Morton 排序，倍數皆相對原生 FP64）

上限	截斷 28 位	混合精度	混合精度 + 截斷 28 位	疊加後再快
1000 W	同速，省電 14.7%（這組原生只有 19–54% 的時間撞上限）	1.62 倍	1.65 倍	1.9%
700 W	+7.4%	1.60 倍	1.71 倍	7.0%
500 W	+7.2%	1.63 倍	1.73 倍	6.1%
對數概似最大偏差	1.3×10^{-4}	0.043	0.043	—

表 5.13　ExaGeoStatCPP 的 tile Cholesky 分解時間比較（ABBA 兩輪平均）

上限	截斷 36 位對複製基準	截斷 28 位對複製基準	截斷 28 位對未修改的原生	SM 時脈（複製基準 → 28 位）
1000 W	快 3.0%	快 4.0%	慢 3.1%	1913 → 1975 MHz
600 W	快 5.0%	快 8.0%	慢 1.3%	1166 → 1305 MHz
450 W	快 5.6%	快 9.4%	慢 0.5%	—

（n = 60000，ABBA 兩輪；Slurm 工作 3244；`gh200/results/gp_mp_summary.csv`）。

截斷可以疊加在改變格式的做法之上：把混合精度剩下的 FP64 tile 再截斷到 28 位，功耗受限時再快 6–7%，偏差不變。把 FP16 運算元也截斷到 5 個尾數位元則沒有額外收益。ExaGeoStatCPP 的混合精度選項（`--computation=mixed`）只有列舉值、沒有實作，所以無法與原版直接比較。這組實驗的位置依 Morton 曲線排序，截斷 28 位在 700 ／ 500 W 快 7.2–7.4%，與表 5.11（未排序）一致。

5.10.4　實際的 ExaGeoStat

我們建置了 ExaGeoStatCPP（StarPU 與 Chameleon 的 tile Cholesky，cuBLAS 13.6，tile 大小 1920），以攔截工具截斷它的 FP64 tile GEMM，執行最大概似估計（N = 60000，每次 6 個 BOBYQA 評估；資料以原生版本產生一次，之後每次都讀取同一份；Slurm 工作 3229；`gh200/results/exageostat_tdp.csv`）。它的 Matérn 共變異數在 CPU 上產生，每次評估約 9 秒，GPU 分解只需 2.3 秒（1000 W）。分解時間的比較（ABBA 兩輪平均）列於表 5.13。

即使在 1000 W，分解也部分受功耗限制（GPU 650–690 W，常有功耗上限造成的降頻事件）。對數概似的相對變化最多為 1.2×10^{-10}（36 位）與 9.8×10^{-8}（28 位）。450 W 的兩輪差異大（截斷 28 位對複製基準為 +6.9% 與 +11.9%，SM 707 MHz 對 926 MHz），因為 CPU 產生共變異數時與 GPU 共用模組上限。攔截工具本身的複製要多花 7–9%，與未修改的 ExaGeoStat 相比仍慢 0.5–3.1%；要取得這個收益，截斷

必須實作於函式庫內（例如 Chameleon 的 GEMM codelet）。

5.10.5　小結

在跨度寬且功耗高的矩陣乘法工作上，整個矩陣使用同一種格式的做法（FP32、INT8 模擬）不是結果錯誤就是較慢；在測試的三種整個矩陣的做法（FP32、cuBLAS 模擬 36 位、截斷）中，只有截斷又快又準，估出的範圍參數也不變。前人的 tile 混合精度快 1.6 倍，截斷還能疊加其上，在功耗受限時再快 6–7%。在實際的 ExaGeoStat 上，截斷使分解比複製基準快 3–9%，但攔截工具的複製成本將其抵銷，因此需要在函式庫內實作。本節的限制是：主要結果來自自寫的核心與合成資料，混合精度變體是簡化的重新實作。

第六章　討論

6.1　適用條件

尾數歸零的效益取決於以下三個條件，缺少其中任何一個，收益就會縮小或消失：

1. **系統受功耗限制**。省下的功耗只有在功耗上限壓住時脈時才會轉成速度。HPL 在 400–800 W 快 4.0–11.0%；調參後的 HPL 在預設的 1000 W 仍有 75% 的時間撞上限，所以也快 3.9%、每次求解少耗電 7.5%。幾乎不受限的程式（例如 1000 W 的 DF-MP2，只有部分時間撞上限）則主要表現為省電，速度只多 0.3–0.8%。
2. **被截斷的資料直接且大量地進入乘法器或資料通道**。矩陣乘法的每個輸入都要參與多次乘法，所以效益最大；N 體計算第一步就做減法，差值的尾數又是滿的；CG 與 SW4 被截斷的資料只佔記憶體流量的一部分；LAMMPS 的位能表常駐快取，幾乎不經過記憶體。
3. **有機制把誤差攔下來**。HPL 的殘差檢查、HPL-MxP 的迭代修正與 SCF 的自洽迭代都會修正或偵測截斷誤差；AI 的權重直接決定輸出品質，截斷便直接轉成精度損失。

表 6.1 彙整各類工作負載在功耗受限時的收益與主要原因。

6.2　與改變格式做法的關係

尾數歸零與 INT8 模擬 FP64、FP32 ／ 3xTF32、FP8 並非互相取代，而是互補。INT8 模擬在所需精度較低（約 28 位以下），或 n 與 k 都達數千以上時較快；截斷則在瘦長矩陣、需要 32 位以上的精度，以及 GEMMul8 這類高功耗模擬遇到嚴格上限時較快。第 5.5 節的三參數公式（式 (5.1)）在測過的範圍內能事先判斷勝負，可以作為函式庫逐次呼叫選擇方法的起點（樣本外案例都離分界很遠，仍需更嚴格的驗證）。FP32 與 3xTF32 的誤差隨內積長度成長，把所有 GEMM 換成它們，在長內積的平面波 DFT 上無法收斂；截斷保留 FP64 累加，誤差與內積長度無關。FP8 已經

表 6.1　各類工作負載在功耗受限時的收益

工作負載	收益	主要原因
具殘差檢查的稠密求解（HPL、HPL-MxP）	HPL 快 4.0–11.0%；HPL-MxP 快 1.8–12.7%（對角優勢測試矩陣）	矩陣乘法主導，殘差檢查把關
高功耗的量子化學（DF-MP2）	快 4.9–8.3%；1000 W 時省電 8–14%	矩陣乘法佔 78%，張量建好後截斷一次即可
跨度寬且功耗高（高斯過程地統計）	快 7.4–8.2%；1000 W 時快 0–4%（依設定）、省電 7–15%	Cholesky 以矩陣乘法為主；FP32 不可用、模擬較慢
自洽迭代（Quantum ESPRESSO）	快 2.4–4.2%（金表面約一半來自迭代減少），總能量印出的每一位都不變	截斷誤差低於當圈的容許誤差
高斯基底量子化學（PySCF）	預設格點估計約 2%（400 W 約 4%）；細格點的纈胺黴素在 450–550 W 實測 3.3–3.5%	矩陣乘法只佔總時間 32%
記憶體受限、係數佔流量多（MFEM、SW4）	快 0.3–2.0%	其餘滿尾數的向量稀釋了效益
係數常駐快取（LAMMPS）	落在雜訊內	被截斷的資料不經過記憶體
已使用 FP8 的 AI 訓練與推論	訓練變慢；推論以精度 1:1 換取速度	FP8 只剩 3 個尾數位元

把 AI 的尾數減到 3 位，在其上再截斷只是以精度換取速度。

整個矩陣改用同一種格式的做法還有一個前提：資料的指數跨度必須夠窄。逐張量縮放的格式（FP32、FP8）取決於整體跨度，逐列切片的 INT8 模擬取決於同一列內的跨度（第 5.9 節）。DF-MP2 的張量跨度窄，所以內建的 FP32 反而比截斷快；高斯基底 DFT 依運算量加權有 24–44% 的元素（最寬的格點原子軌域矩陣達 87%）比所在列的最大值小 2^{-32} 以上，固定位數的模擬便在沒有警告的情況下算錯。不過列內跨度寬只是必要條件（高斯過程的跨度更寬卻沒有算錯），以 tile 為單位選擇格式的混合精度也能處理這類資料。這個跨度可以用攔截工具事先量測，作為風險指標；cuBLAS 的自動模式也會偵測它而改用原生 FP64，但如此便不再比截斷快（第 5.9 節）。

因此，尾數歸零的定位是：在 FP64 科學計算中，一個每個元素只多一次位元運算、精度可以 1 位為單位調整、誤差不受矩陣形狀與指數跨度影響的預設選項。在跨度窄、矩陣大而方、精度需求較低時，改用模擬或 FP32 可能更快；對於能依空間排序把小數值集中到整個 tile 的應用（例如地統計），tile 層級的混合精度能快得更多，截斷則可以疊加在它剩下的 FP64 tile 上。

6.3　限制與效度威脅

1. **單一平台**。所有實驗都在同一個 GH200 叢集上進行；其他 GPU 世代的功耗管理不同，效益大小與分界公式的係數可能改變。

2. **保留位數需以多組問題大小驗證**。HPL 的殘差隨矩陣大小跳動，28 位在三組 N 通過（殘差都接近門檻）、在一組失敗；本報告以有百倍餘裕的 36 位作為操作點。

3. **應用覆蓋有限**。每個科學軟體只測試一到兩個輸入系統，沒有涵蓋幾何最佳化、長時間分子動力學等情境。

4. **攔截工具的複製成本（依應用約 3–9%）**。因此攔截版本一律與「複製但不截斷」的基準比較，只有程式內版本與未修改的原生程式比較。

5. **節點差異**。同一功耗上限下，節點間的 SM 時脈可相差 13–17%，因此所有比較都在同一組節點上成對進行。

6. **散熱條件**。風扇轉速低時 GPU 會先撞到溫度上限；風扇全開時，原生 HPL-MxP 在 1000 W 會觸發機箱的硬體功耗煞車。這些條件都會改變結論，需要一併記錄。

7. **共用叢集**。其他使用者的工具至少兩次重設叢集的功耗上限；我們在每次執行前檢查實際生效的上限，受影響的量測一律重做。

8. **功耗感測器**。功耗讀自驅動程式，沒有外接電表；在 A100 ／ H100 上，驅動程式只取樣部分時間 [44]。我們的比較都是同一量測路徑下的成對相對值，但絕對能量仍需以外接電表驗證。

9. **訓練的收斂點有雜訊**。LoRA 的達標點即使設定完全相同也會相差一個評估區間，而我們只用了兩個亂數種子，所以只把它當作「FP8 沒有再截斷的空間」的佐證。

10. **HPL-MxP 的測試矩陣**。我們使用嚴格對角優勢的矩陣與近似的對角分解，規則只允許這種矩陣用於測試 [42]；「保留 3 位仍穩定」可能只是這個矩陣的性質。

11. **樣本數與節點**。信賴區間只用於矩陣乘法微基準與 Quantum ESPRESSO；HPL、HPL-MxP、DF-MP2 與高斯過程每一格只有兩輪，且在同一組節點上量測。分界點掃描的四個 n 值分別在不同節點上量測，跨上限、跨形狀的趨勢可能混入節點差異。

12. **功耗上限的意義**。1000 W 是預設也是最高上限，並不代表不受限：HPL 與 FP64 矩陣乘法在 1000 W 仍有部分時間撞上限。撞上限時 1 秒平均功耗常低於上限，限電器依據的時間尺度不明；低於 450 W 的上限超出規格表的可設範圍 [41]。

13. **機制實驗的範圍**。填值對照只做了保留 0 位與 20 位，500 W 只在一個節點上量測兩次；保留 0 位時「數值為 0」的額外省電原因尚未驗證。

14. **收益不全來自功耗**。在 Quantum ESPRESSO 金表面上，自動調整的收益約一半來自 Davidson 迭代減少（第 5.4.3 節），原因尚未查明。

6.4　規則合規性

TOP500 的 HPL 要求以完整的 64 位元浮點運算求解，明文排除以低精度求解再迭代修正，以及以整數運算模擬浮點的方法 [45]；截斷運算元後的 HPL 也不符合這個要求。HPL-MxP 自 2025 年 6 月起規定最低精度為 16 位元浮點，且嚴格對角優勢的矩陣只能用於測試 [42]；保留 3 位的 BF16 實質上只有 12 位元，我們的測試矩陣也不符合正式規定。因此本報告所有 HPL ／ HPL-MxP 數字都是研究結果，並非正式成績。MLPerf Inference 的封閉組允許數學上的量化、禁止剪枝，尾數歸零屬於前者 [46]；MLPerf Training 的權重能否截斷需依各項規則判斷，不過就本報告的結果而言，均勻截斷本來就不划算。科學軟體沒有這類限制，判準是物理結果是否在所需的精度之內。

第七章　結論與未來工作

7.1　結論

本專題在現役的 GH200 叢集上，以純軟體將用不到的尾數位元歸零，回答了第 1.4 節的三個問題：

1. **機制：**省電主要來自低位元不再翻轉（佔 62–97%），數值為 0 在乘法器上再多省一些；此結論在 tensor core、一般 FP64 運算單元與記憶體搬移上都成立。

2. **效益：**在功耗受限且截斷實作於程式內時，以矩陣乘法為主、具正確性檢查的完整程式都獲得正面收益：HPL（調參後）保留 36 位快 4.0–11.0%（等效多出 39–127 W），HPL-MxP 保留 3 位快 1.8–12.7%（對角優勢的測試矩陣），Quantum ESPRESSO 依收斂門檻自動調整位元數快 2.4–4.2%、少耗電 2.5–3.9%，總能量印出的每一位都不變；高功耗的 DF-MP2 快 4.9–8.3%，跨度寬且功耗高的高斯過程地統計快 7.4–8.2%。在預設的 1000 W 上限下，HPL 也快 3.9%、每次求解少耗電 7.5%，DF-MP2 少耗電 8–14%。代價是必須以多組問題大小驗證保留位數，並使用最近捨入。

3. **適用範圍：**尾數歸零與 INT8 模擬 FP64 互補，分界點可以用三參數公式事先估計（樣本外 8 個案例全部判斷正確，但案例都離分界很遠）；把所有 GEMM 換成 FP32 ／ 3xTF32 在長內積上無法取代它；列內指數跨度寬時，整個矩陣改變格式可能出錯（逐張量的格式放不下，固定位數的逐列切片模擬可能安靜地算錯），截斷則不受影響，也能疊加在 tile 層級的混合精度之上；在已使用 FP8 的 AI 上，FP8 已取得幾乎全部的好處。非矩陣乘法的程式同樣省電，但在真實軟體中只剩 0.3–2.0% 的收益。

總結而言，在功耗受限的 FP64 科學計算中，尾數歸零是一個每個元素只多一次位元運算、不需要新硬體、精度可以 1 位為單位調整的調整手段，最適合具殘差修正或自洽迭代的求解器。

7.2　未來工作

第二種硬體： 在 H200、MI300A 或 Blackwell 上重做分界點掃描與 HPL，檢驗結論與公式能否移植；HPL-MxP 也需改用規則允許的矩陣產生器與精確的對角分解，重做保留位數的掃描。

函式庫整合： 把「依形狀與上限選擇截斷或模擬」的規則實作於矩陣乘法函式庫（例如 cuBLAS 或 Chameleon 的 GEMM codelet），使截斷不需要額外複製；在 ExaGeoStat 上，複製成本恰好抵銷了收益。

更多自適應求解器： 把「依容許誤差決定位元數」推廣到其他迭代法，例如 Krylov 求解器的內層與混合精度迭代修正；也可以利用資料的位元資訊量事先估計需要保留的位數 [22]，以減少試驗。

端到端比較： 在 GPU4PySCF 中實作依形狀選擇的機制，與 Huang 等人的自適應 INT8 模擬直接比較 [39]。

外接電表： 以外接電表交叉驗證絕對能量。

参考文獻

[1] S. Bhalachandra, B. Austin, S. Williams, and N. J. Wright, “Understanding the impact of input entropy on FPU, CPU, and GPU power,” arXiv preprint arXiv:2212.08805, 2022.

[2] T. Gregersen, P. Patel, and E. Choukse, “Input-dependent power usage in GPUs,” in *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2024, pp. 1872–1877.

[3] H. He, “Strangely, matrix multiplications on GPUs run faster when given “predictable” data! [short],” Thonk From First Principles (Substack blog), https://www.thonking.ai/p/strangely-matrix-multiplications, 2024, published Apr. 29, 2024.

[4] O. Antepara *et al.*, “Benchmark-driven models for energy analysis and attribution of GPU-accelerated supercomputing,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC ’25)*, 2025, pp. 888–904.

[5] J. Lucas and B. Juurlink, “ALUPower: Data dependent power consumption in GPUs,” in *IEEE 24th International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems (MASCOTS)*, 2016, pp. 95–104.

[6] B. Rountree, D. H. Ahn, B. R. de Supinski, D. K. Lowenthal, and M. Schulz, “Beyond DVFS: A first look at performance under a hardware-enforced power bound,” in *IEEE 26th International Parallel and Distributed Processing Symposium Workshops & PhD Forum (IPDPSW)*. IEEE, 2012, pp. 947–953.

[7] Y. Inadomi *et al.*, “Analyzing and mitigating the impact of manufacturing variability in power-constrained supercomputing,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC ’15)*. ACM, 2015, pp. 1–12, article 78.

[8] T. Patki, D. K. Lowenthal, B. Rountree, M. Schulz, and B. R. de Supinski, “Exploring hardware overprovisioning in power-constrained, high performance computing,” in *Proceedings of the 27th International ACM Conference on Supercomputing (ICS ’13)*. ACM, 2013, pp. 173–182.

[9] J. You, J.-W. Chung, and M. Chowdhury, “Zeus: Understanding and optimizing GPU energy consumption of DNN training,” in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, 2023, pp. 119–139. [Online]. Available: https://www.usenix.org/conference/nsdi23/presentation/you

[10] J. Y. F. Tong, D. Nagle, and R. A. Rutenbar, “Reducing power by optimizing the necessary precision/range of floating-point arithmetic,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 8, no. 3, pp. 273–286, 2000.

[11] J. Pool, A. Lastra, and M. Singh, “Energy-precision tradeoffs in mobile graphics processing units,” in *IEEE International Conference on Computer Design (ICCD)*, 2008, pp. 60–67.

[12] ——, “Precision selection for energy-efficient pixel shaders,” in *ACM SIGGRAPH Symposium on High Performance Graphics (HPG)*, 2011, pp. 159–168.

[13] D. Guenther, T. Henriksson, R. Leupers, and G. Ascheid, “Mantissa-masking for energy-efficient floating-point LTE uplink MIMO baseband processing,” in *Design, Automation & Test in Europe (DATE)*, 2016, pp. 1028–1029.

[14] P. V. Argade and D. N. Peroni, “Variable precision computing system,” US Patent 11,455,766 B2 (Advanced Micro Devices, Inc.), filed Sep. 18, 2018, issued Sep. 27, 2022, 2022.

[15] N. Chatterjee *et al.*, “Architecting an energy-efficient DRAM system for GPUs,” in *IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2017, pp. 73–84.

[16] M. O’Connor *et al.*, “Fine-grained DRAM: Energy-efficient DRAM for extreme bandwidth systems,” in *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2017, pp. 41–54.

[17] S. Ghose *et al.*, “What your DRAM power models are not telling you: Lessons from a detailed experimental study,” *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 2, no. 3, 2018.

[18] C. Rubio-González *et al.*, “Precimonious: Tuning assistant for floating-point precision,” in *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis (SC13)*, 2013.

[19] A. Dawson and P. D. Düben, “rpe v5: An emulator for reduced floating-point precision in large numerical simulations,” *Geoscientific Model Development*, vol. 10, pp. 2221–2230, 2017.

[20] N. J. Higham and S. Pranesh, “Simulating low precision floating-point arithmetic,” *SIAM Journal on Scientific Computing*, vol. 41, no. 5, pp. C585–C602, 2019.

[21] A. C. I. Malossi *et al.*, “The transprecision computing paradigm: Concept, design, and applications,” in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2018, pp. 1105–1110.

[22] M. Klöwer, M. Razinger, J. J. Dominguez, P. D. Düben, and T. N. Palmer, “Compressing atmospheric data into its real information content,” *Nature Computational Science*, vol. 1, no. 11, pp. 713–724, 2021.

[23] K. Ozaki, T. Ogita, S. Oishi, and S. M. Rump, “Error-free transformations of matrix multiplication by using fast routines of matrix multiplication and its applications,” *Numerical Algorithms*, vol. 59, no. 1, pp. 95–118, 2012.

[24] H. Ootomo, K. Ozaki, and R. Yokota, “DGEMM on integer matrix multiplication unit,” *The International Journal of High Performance Computing Applications*, vol. 38, no. 4, pp. 297–313, 2024.

[25] K. Ozaki, Y. Uchino, and T. Imamura, “Ozaki scheme II: A GEMM-oriented emulation of floating-point matrix multiplication using an integer modular technique,” arXiv preprint arXiv:2504.08009, 2025.

[26] Y. Uchino *et al.*, “High-performance and power-efficient emulation of matrix multiplication using INT8 matrix engines,” in *SC ’25 Workshops*, 2025.

[27] NVIDIA, “cuBLAS library documentation, section 1.5: Floating point emulation (v13.4; archived 13.2.1),” https://docs.nvidia.com/cuda/cublas/index.html#floating-point-emulation, accessed Oct. 5, 2026.

[28] A. Schwarz *et al.*, “Guaranteed DGEMM accuracy while using reduced precision tensor cores through extensions of the Ozaki scheme,” in *Proceedings of the Supercomputing Asia and International Conference on High Performance Computing in Asia Pacific Region (SCA/HPCAsia)*, 2026, pp. 91–101.

[29] T. Grützmacher, T. Cojean, G. Flegar, H. Anzt, and E. S. Quintana-Ortí, “Acceleration of PageRank with customized precision based on mantissa segmentation,” *ACM Transactions on Parallel Computing*, vol. 7, no. 1, pp. 1–19, 2020.

[30] S. Abdulah, H. Ltaief, Y. Sun, M. G. Genton, and D. E. Keyes, “ExaGeoStat: A high performance unified software for geostatistics on manycore systems,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 29, no. 12, pp. 2771–2784, 2018.

[31] ——, “Geostatistical modeling and prediction using mixed precision tile Cholesky factorization,” in *IEEE 26th International Conference on High Performance Computing, Data, and Analytics (HiPC)*, 2019, pp. 152–162.

[32] S. Abdulah *et al.*, “Accelerating geostatistical modeling and prediction with mixed-precision computations: A high-productivity approach with PaRSEC,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 4, pp. 964–976, 2022.

[33] Q. Cao *et al.*, “Reshaping geostatistical modeling and prediction for extreme-scale environmental applications,” in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2022, pp. 1–12.

[34] J. Ren, H. Ltaief, S. Abdulah, and D. E. Keyes, “Accelerating mixed-precision out-of-core Cholesky factorization with static task scheduling,” in *ISC High Performance 2025 Research Paper Proceedings (40th International Conference)*, 2025, pp. 1–12.

[35] S. Abdulah *et al.*, “Boosting earth system model outputs and saving PetaBytes in their storage using exascale climate emulators,” in *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2024, pp. 1–12.

[36] Q. Cao, S. Abdulah, H. Ltaief, M. G. Genton, D. Keyes, and G. Bosilca, “Reducing data motion and energy consumption of geospatial modeling applications using automated precision conversion,” in *IEEE International Conference on Cluster Computing (CLUSTER)*, 2023, pp. 330–342.

[37] R. Olivares-Amaya, M. A. Watson, R. G. Edgar, L. Vogt, Y. Shao, and A. Aspuru-Guzik, “Accelerating correlated quantum chemistry calculations using graphical processing units and a mixed precision matrix multiplication library,” *Journal of Chemical Theory and Computation*, vol. 6, no. 1, pp. 135–144, 2010.

[38] J. Woo, S. Kim, and W. Y. Kim, “Dynamic precision approach for accelerating large-scale eigenvalue solvers in electronic structure calculations on graphics processing units,” *Journal of Chemical Theory and Computation*, vol. 19, no. 5, pp. 1457–1465, 2023.

[39] H. Huang, W. Shao, and J. Hammond, “Accelerating density fitting with adaptive-precision and 8-bit integer on AI accelerators,” arXiv preprint arXiv:2601.08077, 2026.

[40] W. Dawson, K. Ozaki, J. Domke, and T. Nakajima, “Reducing numerical precision requirements in quantum chemistry calculations,” *Journal of Chemical Theory and Computation*, vol. 20, no. 24, pp. 10 826–10 837, 2024.

[41] NVIDIA, “NVIDIA GH200 Grace Hopper superchip datasheet,” Document 3773000, linked from https://www.nvidia.com/en-us/data-center/grace-hopper-superchip/, Mar. 2025, accessed Oct. 5, 2026.

[42] HPL-MxP, “HPL-MxP rules,” https://hpl-mxp.org/rules.md, accessed Oct. 5, 2026.

[43] M. Fasi, N. J. Higham, M. Mikaitis, and S. Pranesh, “Numerical behavior of NVIDIA tensor cores,” *PeerJ Computer Science*, vol. 7, p. e330, 2021.

[44] Z. Yang, K. Adamek, and W. Armour, “Accurate and convenient energy measurements for GPUs: A detailed study of NVIDIA GPU’s built-in power sensor,” in *Proceedings*

of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC24), 2024.

[45] TOP500.org, "The Linpack benchmark," https://www.top500.org/project/linpack/, accessed Oct. 5, 2026.

[46] MLCommons, "MLPerf inference rules, section 8.1: Weight definition and quantization," https://github.com/mlcommons/inference_policies/blob/master/inference_rules.adoc.

[47] J. Dongarra and P. Luszczek, "HPL-MxP benchmark: Mixed-precision algorithms, iterative refinement, and scalable data generation," *The International Journal of High Performance Computing Applications*, vol. 40, no. 1, pp. 52–62, 2026, published online Sep. 25, 2025; arXiv:2509.19618.

[48] P. Giannozzi *et al.*, "Quantum ESPRESSO toward the exascale," *The Journal of Chemical Physics*, vol. 152, p. 154105, 2020.

[49] X. Wu *et al.*, "Enhancing GPU-acceleration in the Python-based simulations of chemistry frameworks," *WIREs Computational Molecular Science*, vol. 15, no. 2, p. e70008, 2025.

[50] R. Li, Q. Sun, X. Zhang, and G. K.-L. Chan, "Introducing GPU acceleration into the Python-based simulations of chemistry framework," *The Journal of Physical Chemistry A*, vol. 129, no. 5, pp. 1459–1468, 2025.

附錄 A　實驗環境、程式位置與重現方式

A.1　實驗環境

表 A.1　實驗環境與軟體版本

項目	內容
硬體	14 個節點，各一顆 NVIDIA GH200 480GB（aarch64），200 Gb/s InfiniBand，Slurm 分區 `compute`
編譯器與函式庫	NVHPC 26.5（CUDA 13.2、cuBLAS 13、NCCL、HPC-X Open MPI）；ExaGeoStatCPP 連結 cuBLAS 13.6
基準	HPL（rocHPL 的 CUDA 移植，分支 `feat/cuda-port`）、HPL-MxP（分支 `perf/h200-mantissa`） [47]
科學軟體	Quantum ESPRESSO 7.6 [48]、GPU4PySCF 1.8.1 [49, 50]、MFEM（提交 `c362dea`）、SW4（提交 `52b51c9`）、LAMMPS `stable_22Jul2025_update6`、GEMMul8 3.5.1
AI	MLPerf Training Llama 2 70B LoRA，NVIDIA v5.0 實作（8 節點）
分析	Python（pandas、numpy、matplotlib），虛擬環境 `gh200/.venv`

A.2　程式位置

表 A.2 列出本專題的程式與資料位置（路徑相對於專案根目錄）。

表 A.2　程式與資料位置

路徑	用途
`gh200/bench/`	cuBLAS 尾數矩陣乘法微基準（第 5.2、5.5 節）
`gh200/shim/mantshim.cu`	cuBLAS 攔截函式庫：截斷（`MANT_BITS`、`MANT_ROUND`）、cuBLAS 模擬、單精度替換（`MANT_F32`）、cuBLAS 自動模式與位數紀錄（`MANT_EMU_DYN`、`MANT_EMU_LOG`）、運算元統計與傾印
`apps/q-e/KS_Solvers/Davidson/`	Quantum ESPRESSO 的程式內截斷與自動調整規則（`QE_MANT_ADAPT`）
`rocHPL/src/blas/HPL_quantize_device.cpp`	HPL 的運算元捨入（`HPL_QUANT_A_BITS` 等）

表 A.2（續）

路徑	用途
rocHPL-MxP/src_mi350/hplmxp.h	HPL-MxP 寫回 BF16 時的歸零（編譯時指定 MANT_BITS）
apps/（見 apps/建置日誌.md）	MFEM、SW4、LAMMPS 的係數截斷插入點與建置方式
apps/pyscf-inputs/mp2_variants.py	DF-MP2 各變體：程式內截斷 DF 張量、內建 FP32、cuBLAS 模擬（第 5.8 節）
apps/geostat/gp_mle.py	高斯過程對數概似與最大概似估計：截斷、cuBLAS 模擬、FP32、Morton 排序與 tile 混合精度（第 5.10 節）
apps/exageostatcpp/、apps/patches/	ExaGeoStatCPP（submodule）與在 GH200、CUDA 13 上建置所需的修正（建置日誌第 9 節）
paper/	英文 LaTeX 論文稿（main.tex、refs.bib、figs/）
paper/zh_report/	本報告的 LaTeX 原始檔（report.tex、chapters/、figs/）
gh200/refs/	各輪文獻查證紀錄（round4_refs.md、round5_refs.md）
gh200/slurm/	工作腳本：e1_gemm_tdp.sbatch（微基準與形狀掃描）、hpl_tdp.sbatch（HPL ／ HPL-MxP）、sci_sweep.sbatch（科學軟體）、app_tdp.sbatch（第三方軟體）、mp2_tdp.sbatch（DF-MP2）、gp_tdp.sbatch（高斯過程）、exageostat_tdp.sbatch（ExaGeoStat）、hpl_tune.sbatch（HPL 調參）、hpl_shim.sbatch（單 GPU HPL 經攔截工具：逐列跨度與模擬）
gh200/analysis/	summarize.py、sci_summary.py、app_summary.py、crossover.py、qe_adapt_ci.py、range_sweep.py、span_report.py、mp2_summary.py、gp_summary.py、exa_summary.py、paper_figures.py（英文論文的向量圖）、report_figs_zh.py（本報告的圖 5.1–5.13）；圖 5.13 的座標來自 gh200/results/range_map_points.csv
gh200/results/	所有彙整後的 CSV；圖下方、表註與內文列出的結果檔都在此目錄
gh200/實驗日誌.md、gh200/應用實驗日誌.md	完整的實驗過程紀錄（含失敗與重做）

A.3　重現方式

所有工作都以 Slurm 送出；腳本只在自己獨佔的節點上設定功耗上限，結束時恢復為 1000 W，並在每次執行前檢查實際生效的上限。以下為三個例子：

```
module load nvhpc/26.5
cd gh200
# 分界點掃描（第 5.5 節）
```

```
sbatch -w work1 --export=ALL,TDPS="1000 575 450",SHAPES="8192x4096x16384
↪  8192x256x81920",CONFIGS="fp64:52 fp64:36 fp64emu:32 gemmul8:10",WARMUP=2,SECONDS_PER=6
↪  slurm/e1_gemm_tdp.sbatch
.venv/bin/python analysis/crossover.py < 工作編號 ...>
# Quantum ESPRESSO 自動調整位元數（第 5.4 節）
sbatch -w work2 --export=ALL,APP=qe,CASE=si1000,INP=si1000.in,TDPS=550,VARIANTS="native
↪  adapt0:nearest src28:nearest src28:nearest adapt0:nearest native" slurm/sci_sweep.sbatch
.venv/bin/python analysis/sci_summary.py logs/sci/< 工作名>_< 編號>
.venv/bin/python analysis/qe_adapt_ci.py < 彙整 CSV> src28:nearest
# 第三方軟體（第 5.6 節）
sbatch -w work10 --export=ALL,APP=mfem slurm/app_tdp.sbatch
.venv/bin/python analysis/app_summary.py logs/apps/< 工作名>_< 編號>
```

圖下方的資料來源、表註與引用各表的段落列出對應的 Slurm 工作編號與結果檔，原始紀錄可以在 gh200/logs/ 與 gh200/results/ 找到。本報告的圖與 PDF 以下列指令重新產生（在專案根目錄執行；字型約 90 MB，不納入版本控制）：

```
paper/zh_report/fetch_fonts.sh                      # 下載並檢查字型
gh200/.venv/bin/python gh200/analysis/report_figs_zh.py paper/zh_report/fonts   # 圖 5.1–5.13
paper/zh_report/build.sh                            # tectonic (XeLaTeX + BibTeX) → report.pdf
```